Inhaltsverzeichnis

Abbildungsverzeichnis iii

Tabellenverzeichnis iv

Codeverzeichnis v

1 Einleitung 1
- 1.1 Problemstellung . . . 2
- 1.2 Zielsetzung . . . 2

2 Grundlagen zu Phishing 3
- 2.1 Formen und Hintergründe . . . 3
  - 2.1.1 Typische Phishing-Methoden . . . 4
  - 2.1.2 Social Engineering . . . 5
- 2.2 Charakteristische Merkmale . . . 5
  - 2.2.1 Psychologische Merkmale . . . 5
  - 2.2.2 Sprachliche Merkmale . . . 6
  - 2.2.3 Technische Merkmale . . . 8
  - 2.2.4 Beispielanalyse . . . 8
- 2.3 Traditionelle Erkennungsansätze . . . 10
  - 2.3.1 Klassifikation von Phishing-Mails . . . 11
- 2.4 Moderne Erkennungsansätze . . . 12
  - 2.4.1 Maschinelles Lernen . . . 12
  - 2.4.2 Deep Learning . . . 12
  - 2.4.3 Hybride Modelle . . . 13
  - 2.4.4 Herausforderungen und Grenzen . . . 13

3 Natural Language Processing 14
- 3.1 Grundlagen und Entwicklung . . . 14
  - 3.1.1 Vorverarbeitung von Texten . . . 15
  - 3.1.2 Textvektorisierung . . . 16
- 3.2 Moderne Sprachmodelle . . . 18
  - 3.2.1 Transformer-Architektur . . . 19

3.2.2 BERT (Bidirectional Encoder Representations from Transformers) . 21
3.2.3 GPT (Generative Pre-Training Transformer) 25

4 Methodik und Umsetzung **28**
4.1 Datensatzwahl 28
4.2 Evaluationsmetriken 28
4.2.1 Accuracy 29
4.2.2 Precision 29
4.2.3 Recall 30
4.2.4 F1-Score 30
4.2.5 Zusammenfassung der Metriken 30
4.3 Modellimplementierungen 30
4.3.1 Modelinitialisierung 31
4.3.2 Datenvorverarbeitung 31
4.3.3 Trainingsparameter 32
4.3.4 Evaluierungsfunktion 33
4.3.5 DistilBERT zur Erkennung von Mail-Phishing 34
4.3.6 DistilBERT zur Erkennung von SMS-Phishing 36
4.3.7 DistilGPT2 zur Erkennung von Mail-Phishing 38
4.3.8 DistilGPT2 zur Erkennung von SMS-Phishing 40

5 Fazit und Ausblick **42**

Literaturverzeichnis **44**

Abbildungsverzeichnis

2.1 Wortwolke auf Basis von Smishing- und Spam-Nachrichten 7
2.2 Beispiel 1: SMS-Phishing 8
2.3 Beispiel 2: SMS-Phishing 9
2.4 Beispiel 3: Social Engineering über Messenger 9
2.5 Beispiel 4: Mail-Pishing 10

3.1 Embeddings-Visualisierung 18
3.2 Transformer-Architektur 20
3.3 BERT Input Embeddings 22
3.4 Pretraining und Fine-Tuning von BERT 24
3.5 GPT-Modellarchitektur 26

4.1 Trainings-Loss (DistilBERT, Mail-Phishing) 34
4.2 Confusion-Matrix (DistilBERT, Mail-Phishing) 35
4.3 Trainings-Loss (DistilBERT, SMS-Phishing) 36
4.4 Confusion-Matrix (DistilBERT, SMS-Phishing) 37
4.5 Trainings-Loss (DistilGPT2, Mail-Phishing) 38
4.6 Confusion-Matrix (DistilGPT2, Mail-Phishing) 39
4.7 Trainings-Loss (DistilGPT2, SMS-Phishing) 40
4.8 Confusion-Matrix (DistilGPT2, SMS-Phishing) 41

Tabellenverzeichnis

2.1 Verschiedene Arten von Phishing-Angriffen 4
2.2 Cialdinis Prinzipien und deren Anwendung im Phishing 6
2.3 Sprachliche Merkmalskategorien in Phishing-Nachrichten 7
2.4 Technische Merkmale zur Klassifikation von Phishing-Mails 11

3.1 Typische Teilaufgaben im Natural Language Processing 15
3.2 GPT-Modelle im Vergleich 27

4.1 Kontingenztabelle 29
4.2 Evaluationsmetriken (DistilBERT, Mail-Phishing) 35
4.3 Evaluationsmetriken (DistilBERT, SMS-Phishing) 37
4.4 Evaluationsmetriken (DistilGPT2, Mail-Phishing) 39
4.5 Evaluationsmetriken (DistilGPT2, SMS-Phishing) 41

Codeverzeichnis

4.1 Modelinitialisierung DistilBERT 31
4.2 Modelinitialisierung DistilGPT2 31
4.3 Tokenisierung und Aufteilung in Trainings- und Testdaten 32
4.4 Trainingsparameter und Modelltraining 32
4.5 Berechnung der Evaluationsmetriken 33

1 Einleitung

Phishingangriffe gehören zu den ältesten, aber nach wie vor effektivsten Betrugsmethoden im digitalen Zeitalter. Sie zielen auf den potentiell schwächsten Punkt jedes Sicherheitssystems: den Menschen. Diese Angriffe nutzen Emotionen, kognitive Verzerrungen und mangelnde IT-Kompetenz, um sensible Daten zu stehlen oder finanziellen Schaden anzurichten [1].

Die Bedrohungslage für Unternehmen verschärft sich stetig. Laut einer Studie des Digitalverbands Bitkom aus dem Jahr 2024 waren 81 Prozent der Unternehmen in Deutschland im letzten Jahr von Datendiebstahl oder Sabotage betroffen, ein deutlicher Anstieg gegenüber 72 Prozent im Vorjahr. Der wirtschaftliche Schaden stieg um 29 Prozent auf 266,6 Milliarden Euro. Phishing-Attacken stellen dabei mit 26 Prozent der betroffenen Unternehmen die zweithäufigste Ursache dar und verursachten signifikante Schäden [2].

Auch Privatpersonen geraten zunehmend ins Visier von Cyberkriminellen. Laut einer Pressemeldung des Digitalverbands Bitkom aus dem Jahr 2025 haben 61 Prozent der Internetnutzer in Deutschland in den letzten zwölf Monaten persönliche Erfahrungen mit Cyberkriminalität gemacht. Phishing war mit 30 Prozent ebenfalls die zweithäufigste Methode, direkt nach Betrug beim Online-Shopping mit 36 Prozent. Viele Vorfälle werden jedoch nicht gemeldet, da Scham oder der Eindruck, selbst schuld zu sein, viele Menschen davon abhält, Anzeige zu erstatten. Vertrauen, Neugier und Hilfsbereitschaft werden gezielt ausgenutzt, um persönliche Informationen zu erlangen oder finanziellen Schaden zu verursachen. Der durchschnittliche Schaden lag bei den Betroffenen bei 219 Euro [3].

Phishing-Techniken haben sich in den letzten Jahren dramatisch weiterentwickelt. Angreifer setzen zunehmend künstliche Intelligenz ein, um ihre Methoden zu automatisieren und zu verfeinern. Tools wie FraudGPT, ein auf kriminelle Zwecke zugeschnittenes Large Language Model (LLM), erzeugen mit minimalem Aufwand täuschend echte Phishing-Nachrichten, die selbst geschulte Mitarbeiter täuschen können. Das Ziel bleibt jedoch dasselbe: die Manipulation menschlicher Emotionen, um an vertrauliche Informationen zu gelangen [4].

1.1 Problemstellung

Die Erkennung von Phishing-Angriffen wird zunehmend komplexer. Während traditionelle Ansätze wie regelbasierte Filter und Blacklists lange Zeit effektiv waren, stoßen diese Methoden heute zunehmend an ihre Grenzen [5]. Früher ließen sich viele Phishing-Mails anhand typischer Merkmale erkennen, etwa durch unpersönliche Ansprachen, sprachliche Ungenauigkeiten, fehlerhafte Grammatik oder auffällige Betreffzeilen. Cyberkriminelle gehen inzwischen jedoch wesentlich professioneller vor. Moderne Phishing-Kampagnen zeichnen sich durch sprachlich einwandfreie und täuschend echt wirkende E-Mails aus. Tippfehler oder fehlerhafte Umlaute sind nur noch selten ein eindeutiger Hinweis [6].

Klassische Filtermechanismen, die auf festen Regeln und bekannten Mustern basieren, können mit dieser Entwicklung nicht Schritt halten. In den letzten Jahren wurden daher vermehrt Natural Language Processing (NLP)-Modelle zur Erkennung von Phishing-Versuchen entwickelt. Diese Modelle haben das Potenzial, auch subtile Phishing-Versuche durch kontextuelle Analyse von Texten zu erkennen [7].

1.2 Zielsetzung

Ziel dieses Studienprojekts ist es, einen fundierten Überblick über aktuelle Phishing-Methodiken sowie die Grundlagen des Natural Language Processing (NLP) zu geben und darauf aufbauend moderne NLP-Modelle zur automatisierten Erkennung von Phishing-Inhalten systematisch zu untersuchen. Im Fokus stehen dabei die modernen Sprachmodelle BERT und GPT.

Für die Analyse werden geeignete Datensätze ausgewählt, die repräsentative Phishing-Texte enthalten, um eine Evaluation der Modelle zu ermöglichen. Mit diesen Datensätzen werden die ausgewählten Sprachmodelle gezielt nachtrainiert und die Ergebnisse anhand verschiedener Metriken miteinander verglichen und bewertet.

Die Arbeit soll aufzeigen, wie gut aktuelle KI-gestützte Verfahren zur Phishing-Erkennung funktionieren und welche Vorteile sie bieten. Die Ergebnisse machen deutlich, wie die Modelle praktisch genutzt oder weiterentwickelt werden können, um Phishing-Angriffe wirksamer zu erkennen. Abschließend werden die wichtigsten Ergebnisse sowie die Probleme bei der Darstellung komplexer Phishing-Texte in Datensätzen zusammengefasst und ein Ausblick auf zukünftige Forschungsarbeiten gegeben.

2 Grundlagen zu Phishing

In diesem Kapitel werden die zentralen Grundlagen im Bereich Phishing behandelt. Ziel ist es, relevante Begrifflichkeiten zu klären, typische Angriffstechniken zu erläutern sowie charakteristische Merkmale und bestehende Erkennungsmethoden darzustellen. Dadurch wird eine Basis geschaffen, um die in den folgenden Kapiteln vorgestellten Ansätze zur automatisierten Erkennung und Verarbeitung von Phishing-Inhalten besser einordnen zu können. Den Einstieg bildet ein Überblick über die grundlegenden Erscheinungsformen von Phishing-Angriffen.

2.1 Formen und Hintergründe

Der Begriff Phishing leitet sich von den englischen Wörtern password und fishing ab und steht sinnbildlich für das „Angeln“ nach Zugangsdaten. Ursprünglich bezog sich der Begriff auf das gezielte Abgreifen von Passwörtern über täuschend echt gestaltete Webseiten, häufig im Zusammenhang mit Online-Banking oder E-Mail-Diensten. Mit der Weiterentwicklung digitaler Kommunikation und der wachsenden Verfügbarkeit personenbezogener Daten hat sich auch das Phishing verändert. Heute umfasst der Begriff eine Vielzahl an Angriffen über verschiedene Kanäle wie E-Mail, SMS, Telefon oder soziale Netzwerke. In vielen Fällen kann Phishing auch als Einstieg genutzt werden, Systeme zu kompromittieren, Schadsoftware nachzuladen oder kompromittierte Konten für Folgeangriffe zu missbrauchen [8].

Kennzeichnend ist dabei die täuschend echte Aufmachung der Kommunikation sowie die gezielte Ausnutzung menschlicher Routinen, Emotionen und Erwartungen. Die Erfolgsrate solcher Angriffe basiert maßgeblich auf der psychologischen Manipulation der Zielpersonen, wobei oft Dringlichkeit, Angst oder Neugier als treibende Faktoren eingesetzt werden. Besonders kritisch ist dabei, dass moderne Phishing-Angriffe zunehmend personalisiert und damit schwerer erkennbar werden [6].

2.1.1 Typische Phishing-Methoden

Im Laufe der Zeit haben sich zahlreiche Phishing-Varianten herausgebildet, die sich in Kommunikationskanal, Zielgruppe und Angriffsstrategie unterscheiden [9]. Die folgende Tabelle 2.1 gibt einen Überblick über die wichtigsten Methoden und ihre charakteristischen Merkmale:

Tabelle 2.1: Verschiedene Arten von Phishing-Angriffen

Methode	**Beschreibung**
E-Mail-Phishing	Die häufigste Form des Phishings. Nutzer erhalten täuschend echte E-Mails, die zur Eingabe sensibler Daten auf gefälschten Webseiten auffordern oder Schadsoftware enthalten.
Website-Phishing	Gefälschte Webseiten, die über Links in E-Mails, Werbung oder Suchmaschinen verbreitet werden. Die eingegebenen Zugangsdaten werden direkt an Angreifer übermittelt.
Smishing	Phishing über SMS-Nachrichten. Die Nachrichten enthalten in der Regel Links zu gefälschten Webseiten oder fordern das Opfer zur direkten Eingabe vertraulicher Daten auf.
Vishing	Phishing über das Telefon. Angreifer geben sich häufig als Support-Mitarbeiter, Bankangestellte oder Familienangehörige aus, um persönliche Informationen zu erlangen.
Spear-Phishing	Eine gezielte Methode, bei der Angriffe individuell auf bestimmte Personen zugeschnitten sind. Angreifer verwenden zuvor recherchierte persönliche Informationen, um die Glaubwürdigkeit zu erhöhen.
Whaling	Eine spezielle Form des Spear-Phishing, bei der hochrangige Führungskräfte gezielt ins Visier genommen werden, um Zugang zu sensiblen Unternehmensdaten zu erlangen.
Quishing	Phishing über manipulierte QR-Codes. Diese werden z.B. in E-Mails eingebettet und führen nach dem Scannen auf gefälschte Webseiten oder lösen Downloads aus [6].

2.1.2 Social Engineering

Allen genannten Varianten liegt ein gemeinsamer übergeordneter Ansatz zugrunde: Sie basieren auf Methoden des Social Engineerings. Darunter versteht man psychologische Manipulationstechniken, mit denen gezielt menschliche Emotionen ausgenutzt werden, um vertrauliche Informationen zu erlangen oder bestimmte Handlungen herbeizuführen [10].

Ein klassisches Beispiel für Social Engineering ist der sogenannte Enkeltrick. Während früher vorrangig das Telefon genutzt wurde, verlagert sich diese Masche zunehmend auf digitale Kommunikationsdienste. Betrüger geben sich als Verwandte aus und täuschen eine dringende Notlage vor, etwa einen Unfall oder einen finanziellen Engpass. Besonders häufig richten sich solche Angriffe gegen ältere und alleinlebende Personen, die gezielt unter emotionalen Druck gesetzt werden. Typische Einstiegssätze wie „Rate mal, wer hier spricht“ wurden früher am Telefon verwendet, heute greifen Täter vermehrt zu Diensten wie WhatsApp. Dort schreiben sie beispielsweise: „Hallo Mama, hallo Papa, mein Handy ist kaputt, das ist meine neue Nummer...“, geben sich als Kinder oder Enkel aus und drängen auf schnelle Geldüberweisungen. Dieses klassische Betrugsmuster tritt dadurch in abgewandelter Form erneut verstärkt auf und ist aktuell von hoher Relevanz [11].

2.2 Charakteristische Merkmale

Phishing-Nachrichten weisen häufig typische psychologische, sprachliche und technische Merkmale auf, deren Analyse hilft, Angriffe besser zu verstehen und zu erkennen. Dieser Abschnitt gibt einen Überblick über zentrale Merkmale und zeigt typische Beispiele aus der Praxis.

2.2.1 Psychologische Merkmale

Phishing-Angriffe nutzen gezielt psychologische Mechanismen, um Vertrauen aufzubauen oder Druck zu erzeugen. Häufig werden menschliche Emotionen ausgenutzt, um spontanes und unüberlegtes Handeln zu provozieren. Dabei setzen viele Angriffe nicht auf inhaltliche Überzeugung, sondern auf psychologische Reize wie Autorität, Sympathie oder Zeitdruck, die sich anhand der Schlüsselprinzipien der Überzeugung des Sozialpsychologen Cialdini einordnen lassen. Eine Übersicht dieser Prinzipien und ihrer typischen Anwendung im Kontext von Phishing zeigt Tabelle 2.2. Diese wirken häufig unterbewusst und umgehen eine kritische Prüfung der Nachricht durch die betroffene Person [12].

Tabelle 2.2: Cialdinis Prinzipien und deren Anwendung im Phishing [12], [13]

Prinzip	**Typische Wirkung im Phishing**
Reziprozität	Kostenlose Angebote lösen das Gefühl aus, etwas zurückgeben zu müssen, etwa durch das Weitergeben von Informationen.
Konsistenz	Wer bereits einmal zugestimmt hat, neigt eher dazu, weiteren Aufforderungen zu folgen.
Soziale Bewährtheit	Der Verweis auf viele andere Nutzer soll Vertrauen schaffen und zur Nachahmung bewegen.
Sympathie	Freundliche Ansprache oder persönliche Nähe steigern die Kooperationsbereitschaft.
Autorität	Logos, Titel oder offizielle Sprache täuschen Autorität vor und erhöhen die Handlungsbereitschaft.
Knappheit	Künstlicher Zeitdruck soll verhindern, dass kritisch nachgedacht wird.

Diese Prinzipien erklären, warum viele Phishing-Versuche selbst bei technikaffinen Personen erfolgreich sind. Sie zielen auf emotionale Reaktionen ab und umgehen eine bewusste, kritische Bewertung der Situation.

2.2.2 Sprachliche Merkmale

Eine erste Visualisierung zentraler Begriffe aus Smishing- und Spam-Nachrichten ist in Abbildung 2.1 dargestellt. Sie basiert auf dem später verwendeten Datensatz von Mishra et al. [14]. Häufig genannte Wörter wie „free“, „call“ oder „claim“ verweisen auf typische Wörter, mit denen Empfänger gezielt zur Interaktion aufgefordert werden.

Abbildung 2.1: Wortwolke auf Basis von Smishing- und Spam-Nachrichten

Neben einzelnen häufig vorkommenden Schlüsselwörtern lassen sich in Phishing-Nachrichten auch wiederkehrende sprachliche Muster identifizieren. Ein qualitativer Codierungsansatz fasst solche Strukturen systematisch in acht Kategorien zusammen, wie Tabelle 2.3 zeigt, um typische Formulierungen, Themen und Aufforderungen in Phishing-Mails strukturiert erfassen zu können [15].

Tabelle 2.3: Sprachliche Merkmalskategorien in Phishing-Nachrichten [15]

Kategorie	**Beschreibung**
Absender (Organisation)	Namentlich genannte Organisation, z. B. „PayPal“
Absender (Sektor)	Zugeordneter Bereich, z. B. „Finanzen“, „E-Mail“, „Logistik“
Anrede	Persönlich, per E-Mail-Adresse, generisch oder ohne Anrede
Drohende Sprache	Konsequenzen, z. B. „Konto wird gesperrt“, „Datenverlust“
Dringlichkeit	Zeitdruck, z. B. „nur noch 24 Stunden“
Generische Handlung	Aufforderung, z. B. „herunterladen“, „antworten“, „anrufen“
Spezifische Handlung	Aktionszweck, z. B. „Konto verifizieren“, „Passwort ändern“
Hauptthema	Zentrale Nachricht, z. B. „Postfach voll“, „Datei erhalten“

2.2.3 Technische Merkmale

Technische Merkmale betreffen unter anderem verdächtige URLs, gefälschte Absenderadressen oder schadhafte Anhänge. Eine genauere Einteilung solcher Merkmale erfolgt im Abschnitt Klassifikation von Phishing-Mails 2.3.1.

2.2.4 Beispielanalyse

Wie in den vorangegangenen Abschnitten dargestellt, lassen sich Phishing-Nachrichten anhand unterschiedlicher Merkmale systematisch analysieren. Die folgende Auswahl realer Beispiele veranschaulicht typische Angriffsmuster über verschiedene Kommunikationskanäle wie SMS, Messenger und E-Mail.

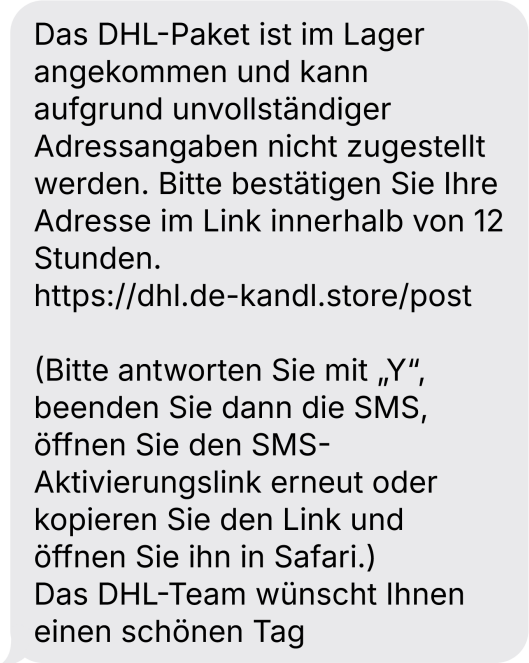

Abbildung 2.2: Beispiel 1: SMS-Phishing

In Abbildung 2.2 wird eine angeblich fehlgeschlagene Paketlieferung vorgetäuscht. Die eingefügte URL verweist auf eine gefälschte Domain, über die sensible Zugangsdaten abgegriffen werden sollen. Der Hinweis auf eine Frist von 12 Stunden erzeugt zusätzlichen Zeitdruck. Auch die Aufforderung zur direkten Antwort ist untypisch für seriöse Versandbenachrichtigungen.

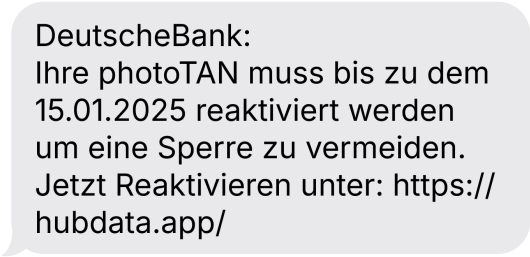

Abbildung 2.3: Beispiel 2: SMS-Phishing

Abbildung 2.3 zeigt eine SMS, in der der Empfänger zur Reaktivierung seiner photoTAN aufgefordert wird. Durch die Angabe einer Frist wird ebenfalls künstlicher Zeitdruck erzeugt. Der URL verweist auf eine gefälschte Domain, über die sensible Zugangsdaten abgegriffen werden sollen.

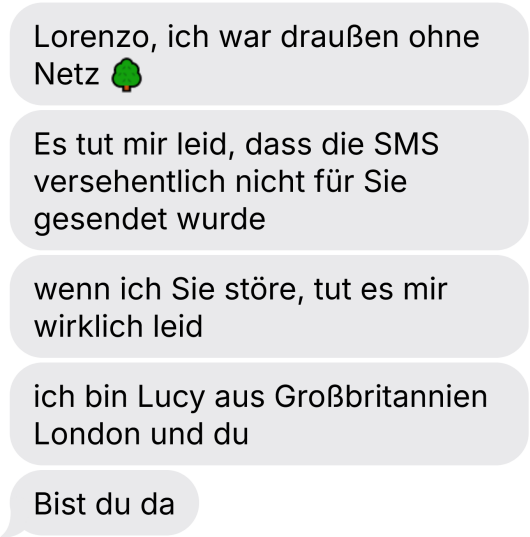

Abbildung 2.4: Beispiel 3: Social Engineering über Messenger

Abbildung 2.4 zeigt einen typischen Social-Engineering-Versuch. Die scheinbar zufällige Kontaktaufnahme über einen Messenger-Dienst soll Vertrauen aufbauen und die Gesprächsbereitschaft erhöhen. Eine konkrete Forderung wird in dieser frühen Phase noch nicht formuliert, sondern schrittweise vorbereitet.

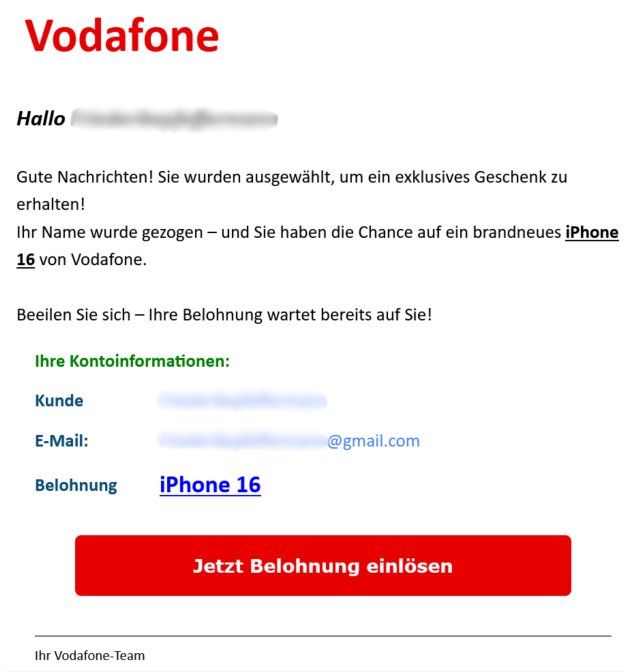

Abbildung 2.5: Beispiel 4: Mail-Pishing

Abbildung 2.5 zeigt eine Phishing-Mail, die durch ein angebliches Gewinnspiel und die vorgespielte Identität eines bekannten Unternehmens Vertrauen aufbauen soll. Die persönliche Ansprache verstärkt die Wirkung, während die Phrase „Beeilen Sie sich“ gezielt Dringlichkeit erzeugt, um die Empfänger zum Klicken auf den Link zu bewegen.

2.3 Traditionelle Erkennungsansätze

Frühe Verfahren zur Erkennung von Phishing basierten hauptsächlich auf heuristischen und regelbasierten Methoden, die auf vordefinierten Mustern wie auffälligen URLs, fehlerhaften Domänennamen oder typischen Schlüsselbegriffen beruhten [6]. Diese Systeme erkannten bekannte Bedrohungen oft zuverlässig, waren jedoch anfällig gegenüber neuen, gezielt angepassten Angriffen, die klassische Regeln bewusst umgingen.

Ergänzend kamen signaturbasierte Ansätze zum Einsatz, wie sie aus der Antivirenerkennung bekannt sind. Hierbei werden E-Mails oder URLs mit Einträgen in einer Datenbank bekannter Phishing-Muster abgeglichen. Gegen neue, abweichende Angriffstechniken bieten diese Verfahren jedoch ebenfalls nur begrenzten Schutz, da sie nicht auf bislang unbekannte Signaturen reagieren können [16].

Die stetige Weiterentwicklung von Phishing-Techniken hat die Probleme dieser statischen Verfahren deutlich aufgezeigt. Um auch neue Angriffe erkennen zu können, bedarf es dynamischer, adaptiver Mechanismen. Einen solchen Übergang markieren merkmalsbasierte Klassifikationsverfahren. Sie analysieren strukturierte Eigenschaften von Nachrichten, sei es regelbasiert oder mithilfe des maschinellen Lernens. Der folgende Abschnitt beschreibt die dabei typischerweise genutzten Merkmalsgruppen [17].

2.3.1 Klassifikation von Phishing-Mails

Ein verbreiteter Ansatz zur automatisierten Phishing-Erkennung ist die merkmalsbasierte Analyse eingehender E-Mails. Spam- und Phishing-Filter extrahieren dabei zahlreiche charakteristische Eigenschaften, um E-Mails zuverlässig klassifizieren zu können. Die Merkmale lassen sich typischerweise in die in Tabelle 2.4 dargestellten technische Kategorien einteilen [5].

Tabelle 2.4: Technische Merkmale zur Klassifikation von Phishing-Mails

Kategorie	**Beschreibung**
Inhaltsbasierte Merkmale	Merkmale aus dem Textkörper der E-Mail, z. B. HTML-Inhalte, bestimmte Phrasen oder eingebettete Hyperlinks. Viele dieser Merkmale lassen sich als binäre Features darstellen.
Betreffzeilen-Merkmale	Eigenschaften der Betreffzeile, etwa Verweise auf frühere Nachrichten oder bestimmte Begriffe wie „verifizieren“ oder „Rechnung“.
URL-bezogene Merkmale	Prüfung auf die Verwendung von IP-Adressen anstelle regulärer Domainnamen, „@“-Symbole in Links, eingebettete Medien sowie interne und externe Verlinkungen.
Skriptbasierte Merkmale	Vorkommen von JavaScript, OnClick-Ereignissen, Pop-up-Fenstern und anderen aktiven Skriptelementen im Quelltext.
Absenderbezogene Merkmale	Analyse der Absenderadresse, insbesondere Abweichungen zwischen angezeigtem Namen und tatsächlichem technischen Absender.

Durch die Kombination verschiedener Merkmale lassen sich verdächtige E-Mails häufig zuverlässig von legitimen Nachrichten unterscheiden. Bei besonders gut getarnten oder gezielt manipulierten Angriffen stoßen solche Systeme jedoch ebenfalls an ihre Grenzen, insbesondere dann, wenn die genannten Merkmale bewusst umgangen werden. Klassische Erkennungsmethoden bieten in solchen Fällen nur eingeschränkten Schutz und können mit der zunehmenden Komplexität moderner Phishing-Strategien oft nicht mehr mithalten. Daraus ergibt sich ein wachsender Bedarf an adaptiven Verfahren, die im nächsten Abschnitt näher erläutert werden [16].

2.4 Moderne Erkennungsansätze

Die folgenden Ausführungen orientieren sich maßgeblich an der Übersicht von Moshood et al. [16] und geben einen kompakten Überblick über moderne Verfahren der künstlichen Intelligenz (KI), die zur Erkennung und Abwehr von Phishing-Angriffen beitragen. Im Fokus stehen dabei Methoden des maschinellen Lernens, Deep Learning und hybride Modelle, die klassische Ansätze bezüglich der Anpassungsfähigkeit übertreffen sollen.

2.4.1 Maschinelles Lernen

Maschinelles Lernen wird zunehmend zur Erkennung von Phishing eingesetzt, da es Muster in Daten erkennt und sich an neue Angriffsmethoden anpassen kann. Überwachte Lernverfahren werden auf Datensätzen mit Phishing- und legitimen Beispielen trainiert, um verdächtige Merkmale zu identifizieren. Häufig genutzte Algorithmen sind Entscheidungsbäume, Support Vector Machines (SVM), k-Nearest Neighbors (KNN) und Random Forests. In vielen Studien liegt der Fokus dabei allerdings klar auf strukturellen Merkmalen wie der URL oder E-Mail-Metadaten und nicht auf inhaltlichen und sprachlichen Mustern [18]. Ein großer Vorteil dieser Verfahren ist ihre Fähigkeit, auch neue, bisher unbekannte Angriffe wie Zero-Day-Phishing zu erkennen.

2.4.2 Deep Learning

Deep Learning, ein Teilbereich des maschinellen Lernens, hat sich als besonders leistungsfähig in der Phishing-Erkennung erwiesen. Im Gegensatz zu klassischen Methoden des maschinellen Lernens erfordern Deep-Learning-Modelle keine manuelle Merkmalsextraktion, da sie relevante Merkmale direkt aus den Rohdaten lernen können. Recurrent Neural Networks (RNNs) und insbesondere LSTM-Netzwerke werden erfolgreich für die Analyse

sequenzieller Textdaten, wie E-Mail-Inhalte, verwendet. Autoencoder ermöglichen darüber hinaus die Erkennung von Anomalien durch das Lernen typischer Kommunikationsmuster. Trotz hoher Genauigkeit erfordern Deep-Learning-Modelle jedoch große Mengen an Daten und hohe Rechenleistung, was ihre Anwendung in Echtzeitumgebungen erschweren kann.

2.4.3 Hybride Modelle

Hybride Ansätze kombinieren mehrere Verfahren der künstlichen Intelligenz, um die Erkennungsrate von Phishing-Angriffen weiter zu verbessern. Dabei werden beispielsweise regelbasierte Methoden mit maschinellem Lernen oder Deep Learning kombiniert, um sowohl bekannte Muster als auch neue, schwer erkennbare Angriffsmethoden abzudecken. Ensemble-Verfahren wie Random Forests kombinieren die Ergebnisse mehrerer Modelle, um Fehler bei der Erkennung zu reduzieren und zuverlässiger zu werden. Hybride Modelle haben sich insbesondere bei der Erkennung komplexer, adaptiver Phishing-Kampagnen bewährt, bringen jedoch oft eine erhöhte Systemkomplexität mit sich.

2.4.4 Herausforderungen und Grenzen

Trotz der Fortschritte durch moderne Verfahren bestehen weiterhin einige Herausforderungen bei der Phishing-Erkennung:

- **Abhängigkeit von Trainingsdaten:** Die Leistungsfähigkeit von KI-Modellen hängt stark von der Qualität und Vielfalt der Trainingsdaten ab. Wenn neue Phishing-Techniken nicht im Trainingsdatensatz enthalten sind, besteht das Risiko, dass diese vom Modell nicht erkannt werden.
- **Hoher Ressourcenbedarf:** Besonders Deep-Learning-Modelle benötigen erhebliche Rechenleistung sowohl beim Training als auch bei der Echtzeitanalyse. Dies kann den Einsatz solcher Modelle für Organisationen mit begrenzter technischer Infrastruktur enorm einschränken.
- **Eingeschränkte Kontextverarbeitung:** Einige Phishing-Angriffe nutzen gezielte, kontextabhängige Täuschungsversuche, wie etwa personalisierte Spear-Phishing-Attacken. Aktuelle Modelle haben Schwierigkeiten, solche komplexen Zusammenhänge zuverlässig zu erkennen, insbesondere wenn zusätzlicher Kontext fehlt.

Angesichts dieser Herausforderungen gewinnt die Analyse natürlicher Sprache zunehmend an Bedeutung.

3 Natural Language Processing

Dieses Kapitel gibt einen Überblick über die Grundlagen des Natural Language Processing (NLP) und stellt ausgewählte Modellarchitekturen vor, die zur Klassifikation von Phishing-Texten eingesetzt werden können.

3.1 Grundlagen und Entwicklung

Natural Language Processing (NLP) bezeichnet im weitesten Sinne die automatisierte Verarbeitung und Analyse von Sprache durch Computersysteme. Ziel von NLP ist es, Sprache so zu verarbeiten, dass sowohl deren strukturelle Eigenschaften als auch deren inhaltliche Bedeutung maschinell ausgewertet werden können [7].

Im Laufe der letzten Jahrzehnte haben sich verschiedene methodische Ansätze im Bereich des NLP entwickelt, die sich grob in drei Entwicklungsphasen einteilen lassen.

- **Regelbasierte Verfahren (1950er–1980er):** In den Anfangsjahren dominierten Ansätze, bei denen Sprache mit Hilfe fester Regeln und Wörterbücher verarbeitet wurde. Viele dieser Systeme arbeiteten rein regelbasiert [19].
- **Statistische Verfahren (ab den 1990er-Jahren):** Mit wachsender Verfügbarkeit von digitalen Textsammlungen kamen datengetriebene Verfahren auf. Statt Regeln zu definieren, wurden Sprachmuster anhand von Wahrscheinlichkeiten modelliert, wie zum Beispiel beim Candide-System von IBM [20].
- **Neuronale Netze und Transformer-Modelle (seit ca. 2015):** Moderne Sprachmodelle wie BERT oder GPT lernen aus großen Textmengen, den Zusammenhang von Wörtern besser zu verstehen. Dadurch lassen sich viele Aufgaben der Textverarbeitung flexibler lösen [21]. In dieser Arbeit werden beide Modelltypen zur automatischen Klassifikation von Phishing-Nachrichten anhand ihres Textinhalts eingesetzt.

3.1.1 Vorverarbeitung von Texten

Vor der eigentlichen Klassifikation von Texten erfolgt im NLP eine Reihe von Vorverarbeitungsschritten. Diese dienen dazu, Rohtexte in eine Form zu bringen, die für maschinelle Lernverfahren effizient verarbeitbar ist. Dazu gehören sowohl grundlegende Umwandlungen wie die Zerlegung von Texten in Wörter als auch Verfahren zur Reduktion von Datenkomplexität. Tabelle 3.1 fasst typische Vorverarbeitungsschritte zusammen [22].

Tabelle 3.1: Typische Teilaufgaben im Natural Language Processing

Aufgabe	Beschreibung
Tokenisierung	Zerlegung von Texten in einzelne Wörter oder Zeichen in Tokens, um diese maschinell verarbeiten zu können.
Entfernung von Stoppwörtern	Entfernung häufiger, wenig informativer Wörter, z. B. „und“, „ist“ oder „ein“, um den Fokus auf bedeutungsstärkere Wörter zu lenken.
Lemmatisierung und Stemming	Reduktion von Wörtern auf ihre Grundform oder Wortstamm, wodurch verschiedene Wortformen als identisch behandelt werden, z. B. „läuft“, „lief“ → „laufen“.
Vektorisierung	Umwandlung von Texten in numerische Repräsentationen, z. B. TF-IDF oder Word2Vec, um eine Weiterverarbeitung durch maschinelle Lernverfahren zu ermöglichen.

3.1.1.1 Data Augmentation mit Textdaten

Im Bereich der Computer Vision ist Data Augmentation weit verbreitet, um aus einem bestehenden Datensatz zusätzliche Trainingsbeispiele zu generieren, beispielsweise durch Bildrotationen oder -spiegelungen. Bei Textdaten gestaltet sich Data Augmentation jedoch deutlich schwieriger, da schon kleine Anpassungen den Sinn oder die Verständlichkeit des Textes stark verändern können.

Trotz dieser Herausforderungen gibt es einige Ansätze, um Textdaten künstlich zu erweitern. Ein häufig eingesetztes Verfahren ist Easy Data Augmentation (EDA), das auf folgenden Techniken basiert [23]:

- **Synonym Replacement (SR):** Zufällige Auswahl von n Wörtern (ohne Stoppwörter), die durch zufällig gewählte Synonyme ersetzt werden.
- **Random Insertion (RI):** Ein zufälliges Synonym eines zufällig gewählten Wortes wird an zufälliger Position eingefügt, n-mal wiederholt.
- **Random Swap (RS):** Zwei zufällig gewählte Wörter werden in ihrer Position vertauscht, n-mal wiederholt.
- **Random Deletion (RD):** Jedes Wort wird mit einer definierten Wahrscheinlichkeit p zufällig entfernt.

In dieser Arbeit wurde auf solche Ansätze verzichtet, da die vorliegenden Datensätze eine ausreichende Menge an Originaldaten enthielten und somit keine künstliche Erweiterung notwendig war.

3.1.2 Textvektorisierung

Für die allgemeine Verarbeitung von Textdaten sowie auch für die inhaltsbasierte Phishing-Erkennung ist es besonders wichtig, dass Wörter unterschiedlich gewichtet werden. Phishing-Nachrichten enthalten oft bestimmte Begriffe, die besonders typisch oder verdächtig sind. Eine geeignete Gewichtung unterstützt dabei, relevante sprachliche Merkmale besser sichtbar zu machen und für die Klassifikation hervorzuheben [22].

Damit maschinelle Lernverfahren Textdaten verarbeiten können, müssen diese in numerische Vektoren dargestellt werden. Zu den klassischen Verfahren zählen TF-IDF, das auf statistischen Häufigkeiten basiert, sowie Word2Vec, das auf der gemeinsamen Verwendung von Wörtern im Kontext beruht. Beide Methoden erzeugen kontextunabhängige statische Wortrepräsentationen.

3.1.2.1 TF-IDF: Frequenzbasierte Wortgewichtung

Term Frequency–Inverse Document Frequency (TF-IDF) ist eine statistische Methode zur Gewichtung von Wörtern, basierend auf ihrer Häufigkeit in einem einzelnen Text und ihrer Seltenheit in der gesamten Textsammlung. Die Berechnung erfolgt durch die Kombination zweier Komponenten [22]:

Term Frequency (TF): Gibt an, wie häufig ein Begriff t in einem einzelnen Text d vorkommt:

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{3.1}$$

Dabei bezeichnet $f_{t,d}$ die absolute Häufigkeit des Begriffs t im Text d.

Inverse Document Frequency (IDF): Bewertet, wie selten ein Begriff t in der gesamten Textsammlung D ist:

$$\mathrm{idf}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right) \tag{3.2}$$

Hierbei ist N die Anzahl aller Texte und $\mathrm{df}(t)$ die Anzahl der Texte, in denen der Begriff t vorkommt. Der Logarithmus sorgt dafür, dass sehr seltene Begriffe nicht übergewichtet werden.

Die endgültige TF-IDF-Gewichtung ergibt sich durch die Multiplikation beider Werte:

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t) \tag{3.3}$$

Da TF-IDF keine semantischen Zusammenhänge berücksichtigt, ist die Methode bei manchen Phishing-Texten möglicherweise weniger effektiv [7].

3.1.2.2 Word2Vec: Statische Wort-Embeddings

Word2Vec erzeugt sogenannte Wort-Embeddings, indem es jedem Wort einer Textsammlung einen Vektor im mehrdimensionalen Raum zuordnet. Diese Vektorrepräsentationen bilden die Bedeutung von Wörtern ab, indem sie deren Verwendung im Kontext berücksichtigen. Wörter, die in ähnlichen Zusammenhängen auftreten, werden durch Vektoren dargestellt, die im Raum nahe beieinanderliegen. Zur Berechnung der Nähe zwischen den Vektoren wird typischerweise die Kosinus-Ähnlichkeit verwendet, welche den Winkel zwischen zwei Vektoren misst [7]:

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \cdot \|\vec{B}\|} \tag{3.4}$$

Dadurch lassen sich Beziehungen zwischen Wörtern ausdrücken, beispielsweise:

- „König" - „Mann" + „Frau" ≈ „Königin"
- „Lissabon" - „Portugal" + „Frankreich" ≈ „Paris"

Word2Vec erfasst somit nicht nur die Bedeutung einzelner Wörter, sondern auch deren Zusammenhänge im Vektorraum [7]. Eine beispielhafte Visualisierung solcher Wortbeziehungen zeigt Abbildung 3.1.

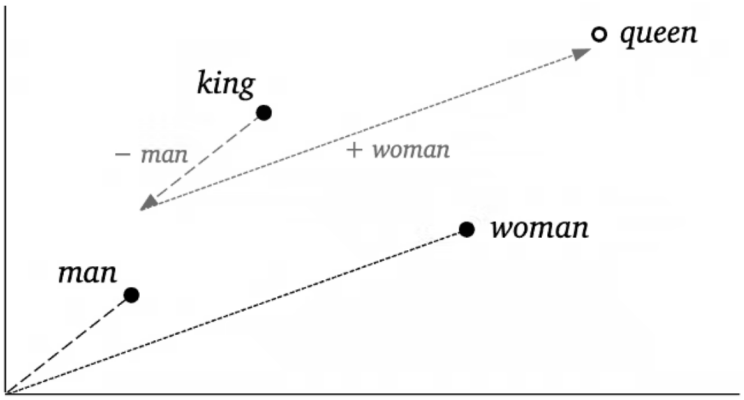

Abbildung 3.1: Embeddings-Visualisierung, übernommen aus [24]

Trotz der Fortschritte durch Word2Vec handelt es sich hierbei weiterhin um statische Wort-Embeddings: Ein Wort erhält immer dieselbe Vektorrepräsentation, unabhängig vom Kontext. Um diese Einschränkung zu überwinden und kontextabhängige Wortbedeutungen modellieren zu können, wurden moderne Sprachmodelle auf Basis der Transformer-Architektur entwickelt.

3.2 Moderne Sprachmodelle

Die folgenden Ausführungen zur Transformer-Architektur orientieren sich maßgeblich an der ursprüngliche Veröffentlichung von Vaswani et al. [25].

Mit der Einführung der Transformer-Architektur wurde die Basis für eine neue Generation von Sprachmodellen geschaffen. Diese Modelle dienen dazu, Eingabetexte in Ausgaben umzuwandeln, etwa um Texte zu übersetzen, Fragen zu beantworten oder Texte zu vervollständigen. Sie sind besonders flexibel und leistungsfähig bei der natürlichen Sprachverarbeitung.

Dies zeigt sich besonders deutlich bei modernen Large Language Models (LLMs) wie den GPT-Modellen. Sie verfügen teils über hunderte Milliarden Parameter und wurden auf riesigen Textmengen trainiert. Dadurch können sie eine Vielzahl von Sprachaufgaben zuverlässig lösen und haben in kürzester Zeit weltweit enorme Popularität erlangt [26].

3.2.1 Transformer-Architektur

Im Gegensatz zu älteren Modellen, die auf Recurrent Neural Networks (RNN) oder Convolutional Neural Networks (CNN) basieren, nutzt der Transformer sogenannte Self-Attention-Mechanismen, wodurch Zusammenhänge und Relevanz zwischen Wörtern unabhängig von ihrer Position erkannt werden können. Außerdem kann das Modell dadurch deutlich schneller trainiert werden und eignet sich besonders gut für die parallele Verarbeitung großer Datenmengen.

Der Transformer verwendet eine sogenannte Encoder-Decoder-Architektur, wie in Abschnitt 3.2.1.2 näher erläutert. Dabei nimmt der Encoder eine Eingabesequenz $x = (x_1, \ldots, x_n)$ auf und wandelt sie in eine Zwischenrepräsentation $z = (z_1, \ldots, z_n)$ um, die die wesentlichen Informationen der Eingabe zusammenfasst. Der Decoder greift auf diese Repräsentation zu, um schrittweise eine Ausgabesequenz $y = (y_1, \ldots, y_m)$ zu erzeugen. Dabei hängt jedes neue Wort auch von den zuvor generierten Wörtern ab.

3.2.1.1 Self-Attention

Das Kernelement des Transformers ist die Self-Attention. Für eine Eingabesequenz werden für jedes Wort sogenannte Query (Q), Key (K) und Value (V) Vektoren berechnet. Dies ermöglicht es dem Modell, die Bedeutung eines Wortes im Zusammenhang mit den anderen Wörtern in der Sequenz zu erfassen. Jedes Wort wird dabei abhängig davon unterschiedlich gewichtet. Die Attention wird folgendermaßen berechnet [25]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{3.5}$$

Hierbei steht d_k für die Dimension der Key-Vektoren. Der Ausdruck $QK^\top$ misst die Übereinstimmung zwischen den Wörtern im Satz, die Softmax-Funktion normiert diese Ähnlichkeitswerte zu Wahrscheinlichkeiten, und mit V wird schließlich der neue Wortvektor berechnet.

3.2.1.2 Encoder und Decoder

Der Encoder besteht aus mehreren identisch aufgebauten Schichten. Jede Schicht kombiniert eine Self-Attention-Schicht, die Zusammenhänge zwischen allen Wörtern der Eingabe berechnet, mit einem kleinen Feed-Forward-Netzwerk, das jedes Wort unabhängig wei-

terverarbeitet. Der Decoder hat einen ähnlichen Aufbau, ergänzt jedoch eine zusätzliche Schicht, die gezielt Informationen aus dem Encoder nutzt. Eine Maskierung innerhalb des Decoders sorgt dafür, dass während der Generierung eines Tokens ausschließlich vorherige Positionen berücksichtigt werden und ein Zugriff auf nachfolgende Tokens unterbunden wird.

Eine Übersicht der Architektur zeigt Abbildung 3.2.

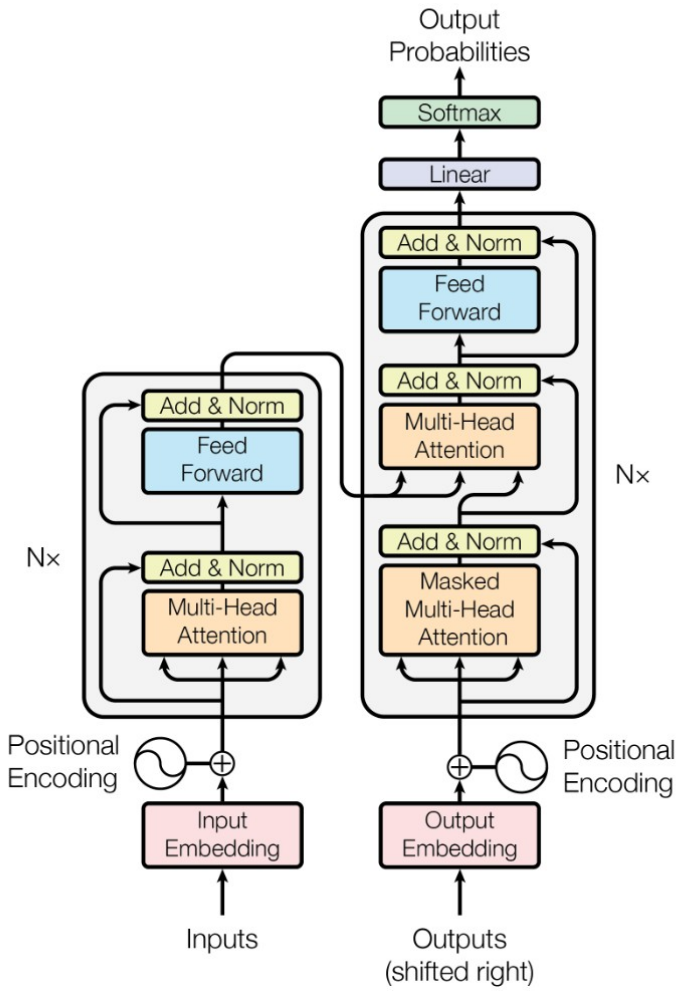

Abbildung 3.2: Transformer-Architektur, übernommen aus [25]

3.2.1.3 Embeddings und Softmax

Damit das Modell Wörter verarbeiten kann, werden diese zunächst durch Embeddings in Vektoren der Dimension d_{model} umgewandelt. Die gleichen Gewichtungen werden hier sowohl für die Eingabe als auch die Ausgabe verwendet, wodurch die Anzahl der Modellparameter reduziert wird. Am Ende des Decoders wird eine Softmax-Schicht genutzt, um Vorhersagen über das nächste Wort in der Ausgabesequenz zu treffen.

3.2.1.4 Positionskodierung

Da der Transformer keine Informationen über die Reihenfolge von Wörtern besitzt, werden Positionsinformationen explizit hinzugefügt. Die sogenannte Positional Encoding fügt jedem Wort eine zusätzliche Information hinzu, die auf sinus- und cosinus-basierten Funktionen beruht, abhängig von der Position *pos* und der Dimension i [25]:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{3.6}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \tag{3.7}$$

Damit kann der Transformer sowohl Wortbedeutung als auch Positionsinformationen verarbeiten, ohne rekursive Strukturen zu verwenden und bildet somit die Grundlage vieler moderner Sprachmodelle wie BERT, GPT und weiterer LLMs.

3.2.2 BERT (Bidirectional Encoder Representations from Transformers)

Die Beschreibung von BERT und seinen Trainingsmechanismen basiert inhaltlich auf der Veröffentlichung von Devlin et al. [21].

BERT ist ein vortrainiertes Sprachmodell, das auf der Encoder-Architektur des Transformers basiert und 2018 von Forschern bei Google vorgestellt wurde. Es zählt zu den ersten Modellen, die kontextualisierte, dynamische Wortrepräsentationen erzeugen. Mithilfe von Self-Attention und Feed-Forward-Schichten werden die Eingabetokens in jeder Ebene aufeinander bezogen, sodass die Bedeutung eines Tokens im Zusammenhang mit allen anderen Tokens der Eingabesequenz modelliert werden kann. Das Ergebnis sind kontextabhängige Repräsentationen, die die jeweilige Wortbedeutung deutlich besser erfassen, als

frühere statische Verfahren wie Word2Vec. Bei seiner Einführung erzielte BERT erhebliche Leistungsgewinne auf zahlreichen NLP-Benchmarks und dient mit seiner Architektur als Grundlage für viele nachfolgende Modelle.

3.2.2.1 Modellarchitektur

BERT besteht aus mehreren Komponenten, die zusammen eine reine Encoder-Architektur gemäß dem zuvor beschriebenen Transformer-Design bilden. Der gesamte Aufbau lässt sich in vier Teile gliedern:

- **Tokenizer:** Zerlegt den Eingabetext in Subwörter (WordPiece) und wandelt diese in numerische Token-IDs um.
- **Embedding:** Wandelt die Token-IDs in Vektoren um und ergänzt Positions- sowie Segmentinformationen.
- **Encoder:** Besteht aus mehreren Schichten mit Self-Attention und Feedforward-Netzen zur Berechnung kontextabhängiger Repräsentationen.
- **Task Head:** Verarbeitet die Encoder-Ausgaben weiter und erzeugt je nach Aufgabe z. B. Klassenzugehörigkeiten oder Wahrscheinlichkeiten für Tokens.

3.2.2.2 Embeddings

Die Eingabe in BERT besteht aus drei Komponenten: Token-Embeddings, Segment-Embeddings und Positions-Embeddings. Diese werden addiert, um den finalen Eingabevektor für jedes Token zu erzeugen. Abbildung 3.3 zeigt den Aufbau dieser Eingaberepräsentation anhand eines Beispielinputs.

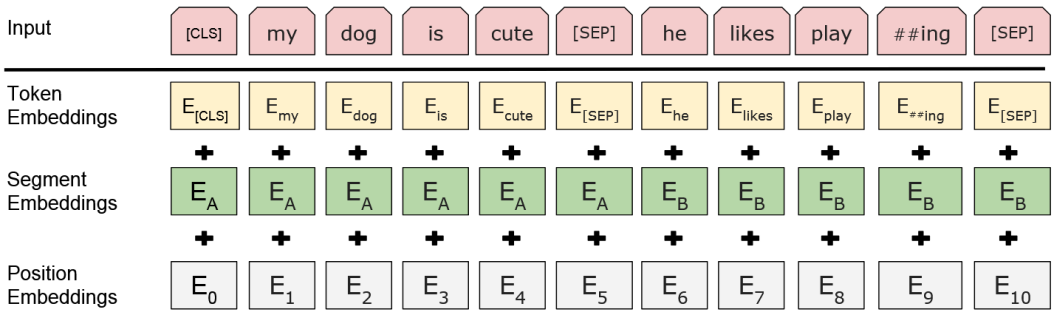

Abbildung 3.3: BERT Input Embeddings, übernommen aus [21]

Am Anfang jeder Eingabesequenz wird das `[CLS]`-Token eingefügt, dessen Ausgabewert später für Klassifikationsaufgaben genutzt wird. Um zwei Sätze innerhalb einer Eingabe voneinander zu trennen, verwendet BERT das `[SEP]`-Token. Darüber hinaus kommen Segment-Embeddings zum Einsatz, um zwischen Satz A und Satz B zu unterscheiden. Positions-Embeddings ergänzen die Eingabevektoren um Informationen zur Reihenfolge der Tokens.

3.2.2.3 Pre-Training

Dem eigentlichen Einsatz von BERT geht ein Pre-Training voraus, das sich in zwei Aufgaben unterteilen lässt:

- **Masked Language Modeling (MLM):** 15 % der Tokens werden maskiert, und das Modell soll sie anhand des Kontexts vorhersagen. Dabei werden 80 % durch `[MASK]` ersetzt, 10 % durch Zufallstokens und 10 % unverändert belassen, um das Modell nicht zu sehr auf das Maskierungssymbol zu fixieren.
- **Next Sentence Prediction (NSP):** Das Modell entscheidet, ob ein zweiter Satz tatsächlich auf den ersten folgt („IsNext“) oder zufällig gewählt wurde („NotNext“). So wird das Verständnis von Satzbeziehungen trainiert.

Für das Training wurden der BooksCorpus mit rund 7.000 Büchern und die englischsprachige Wikipedia mit etwa 2,5 Milliarden Wörtern verwendet. Berücksichtigt wurde dabei ausschließlich zusammenhängender Fließtext.

3.2.2.4 Fine-Tuning

Nach dem Pretraining kann BERT durch Fine-Tuning für konkrete Aufgaben angepasst werden, etwa zur Textklassifikation oder zur Beantwortung von Fragen. Während das Pretraining auf großen Mengen Texte ein allgemeines Sprachverständnis vermittelt, erfolgt das Fine-Tuning auf kleineren, gelabelten Datensätzen. Dabei kommt häufig ein einfacher Klassifikator zum Einsatz, beispielsweise eine zusätzliche lineare Schicht. Anschließend wird das gesamte Modell für wenige Epochen weitertrainiert. Da BERT bereits über ein tiefes Sprachverständnis verfügt, reicht dieses gezielte Nachtrainieren oft aus, um sehr gute Ergebnisse bei spezifischen Aufgaben zu erzielen.

Abbildung 3.4 zeigt den Ablauf des Pretrainings und Fine-Tunings schematisch. Links ist das Pretraining mit Masked Language Modeling (MLM) und Next Sentence Prediction

(NSP) zu sehen, rechts das Fine-Tuning mit konkreten Aufgaben wie SQuAD (Stanford Question Answering Dataset), NER (Named Entity Recognition) und MNLI (Multi-Genre Natural Language Inference).

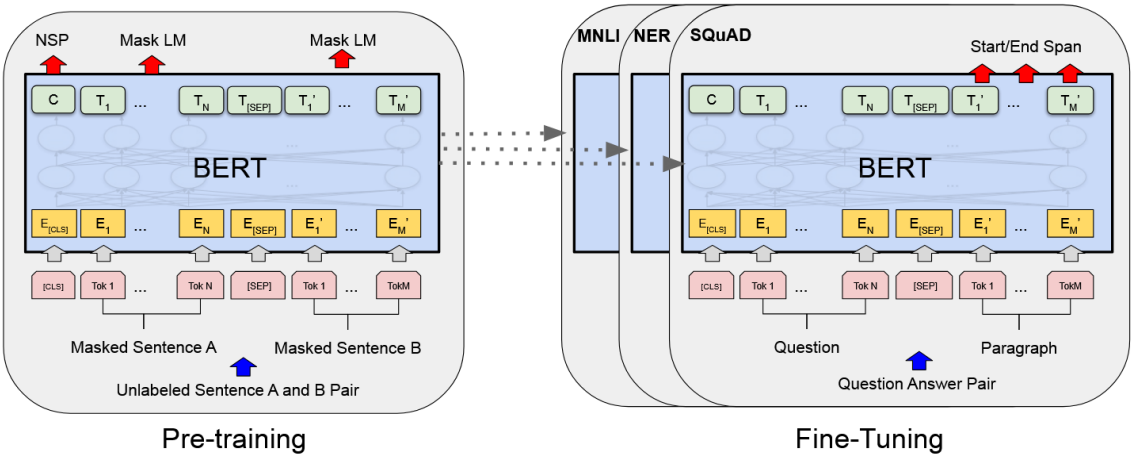

Abbildung 3.4: Pretraining und Fine-Tuning von BERT, übernommen aus [21]

3.2.2.5 Varianten

Seit der Einführung von BERT wurden zahlreiche Varianten entwickelt, die Architektur, Trainingsmenge oder Trainingsstrategie modifizieren. Im Folgenden werden einige ausgewählte Varianten vorgestellt:

- **RoBERTa:** Durch robuste Trainingsoptimierungen stellt RoBERTa eine Weiterentwicklung von BERT dar. Im Gegensatz zum Original verzichtet das Modell auf NSP, verwendet dynamisches Maskieren anstelle von statischem, nutzt größere Mini-Batches und wird auf deutlich mehr Textdaten trainiert. Diese Änderungen ermöglichen eine bessere Ausnutzung des Masked Language Modeling-Ansatzes und führen zu höheren Genauigkeiten auf Benchmarks wie GLUE und SQuAD [27].

- **DistilBERT:** Entwickelt wurde dieses Modell als komprimierte und effizientere Variante von BERT, die mithilfe von Knowledge Distillation trainiert wurde. Dabei diente das ursprüngliche BERT-Base-Modell als Lehrmodell. DistilBERT wurde auf denselben Textdaten vortrainiert und nutzt ein reduziertes Architekturdesign mit halbierter Anzahl an Transformer-Layern. Trotz 40% weniger Parametern erreicht DistilBERT etwa 97% der ursprünglichen Sprachverständnisleistung, bei rund 60% kürzerer Antwortzeit bei Vorhersagen, was es insbesondere für ressourcenbeschränkte Anwendungen attraktiv macht [28].

- **ALBERT:** Bei dieser Variante (A Lite BERT) lag der Fokus auf einer drastischen Reduktion der Modellparameter, ohne signifikante Leistungseinbußen. Zwei zentrale Techniken ermöglichen dies: Parameter-Sharing, bei dem die Gewichte über alle Transformer-Layer hinweg geteilt werden, und die sogenannte Factorized Embedding Parameterization, welche die Größe des Vokabular-Embeddings von der Hidden-Size des Modells trennt. Dadurch sinkt der Speicherbedarf erheblich [29].

3.2.3 GPT (Generative Pre-Training Transformer)

Die Beschreibung von GPT basiert inhaltlich auf den Veröffentlichungen von Radford et al. [30].

GPT beschreibt eine Modellreihe autoregressiver Sprachmodelle, die von OpenAI entwickelt wurden und in den letzten Jahren einen regelrechten Hype ausgelöst haben. Im Gegensatz zu BERT, das ausschließlich auf der Encoder-Architektur des Transformer-Modells basiert, verwendet GPT nur die Decoder-Komponente. Dadurch verarbeiten sie Texte nur von links nach rechts. Ziel ist es, durch generatives Pretraining auf großen Mengen Textdaten ein Sprachverständnis zu entwickeln, das anschließend durch Fine-Tuning auf konkrete Aufgaben wie Textklassifikation übertragen werden kann.

3.2.3.1 Modellarchitektur

Die Grundlage der GPT-Reihe bildet die Architektur von GPT-1. Dieses Modell besteht aus einem 12-schichtigen Transformer-Decoder mit maskierter Self-Attention. In jeder Schicht werden 768-dimensionale Zustandsvektoren verarbeitet, wobei 12 Attention-Heads parallel unterschiedliche Aspekte der Eingabesequenz analysieren können. Diese ermöglicht es dem Modell, vielfältige Kontextinformationen zu erfassen. Die Maskierung in der Self-Attention bewirkt, dass ein Token nur Informationen vorheriger Positionen berücksichtigen kann. Insgesamt umfasst GPT-1 etwa 117 Millionen Parameter.

Abbildung 3.5 zeigt das grundlegende Architekturprinzip von GPT.

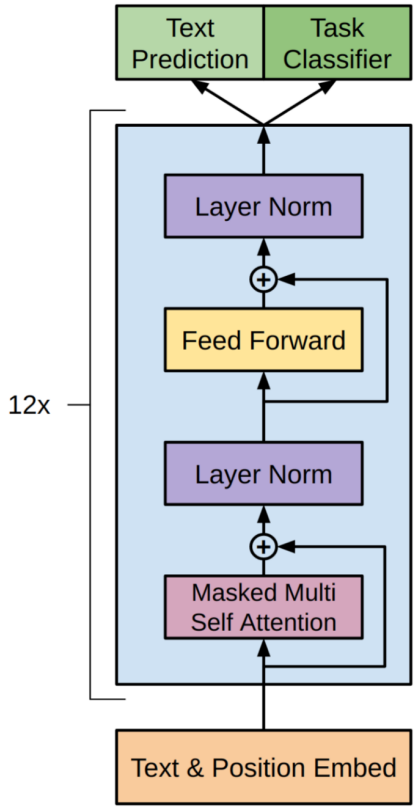

Abbildung 3.5: GPT-Modellarchitektur, übernommen aus [30]

Das Nachfolgemodell GPT-2, das im Rahmen dieser Arbeit für die Phishing-Klassifizierung eingesetzt wird, umfasst 48 Transformer-Schichten, 1600-dimensionale Zustände und rund 1,5 Milliarden Parameter. Die Kontextlänge wurde auf 1024 Tokens erweitert. Strukturell bleibt GPT-2 bei dem autoregressiven Decoder-Prinzip [31].

3.2.3.2 Pre-Training

Beim Pretraining lernt das Modell, das jeweils nächste Token auf Basis des bisherigen Kontexts vorherzusagen. GPT-1 wurde, ebenso wie BERT, mit dem BooksCorpus trainiert, einer Sammlung von über 7000 vollständigen Büchern. Für GPT-2 wurde ein deutlich größerer Datensatz verwendet. Das sogenannte WebText-Korpus enthält rund 8 Millionen qualitativ hochwertige Texte aus dem Internet und soll ein breiteres Sprachverständnis ermöglichen [31].

3.2.3.3 Fine-Tuning

Nach dem Pretraining können GPT-Modelle ebenfalls durch Fine-Tuning an konkrete Aufgaben angepasst werden. Wie im Abschnitt 3.2.2.4 zu BERT erläutert, bleibt auch hier die Architektur unverändert. Lediglich eine Klassifikationsschicht wird ergänzt. Anschließend wird das Modell auf einem kleineren, gelabelten Datensatz weitertrainiert, um relevante Muster für die jeweilige Zielaufgabe zu erlernen. Auch bei GPT genügt in der Regel ein gezieltes Nachtrainieren über wenige Epochen, um gute Ergebnisse zu erzielen.

3.2.3.4 Modellentwicklung

Seit der Veröffentlichung von GPT-1 hat sich die Modellfamilie rasant weiterentwickelt. Tabelle 3.2 gibt einen Überblick über zentrale Versionen und deren zunehmende Modellgröße.

Tabelle 3.2: GPT-Modelle im Vergleich, übernommen aus [32]

Modell	**Jahr**	**Parameteranzahl**	**Trainingsdaten**
GPT-1	2018	117 Mio.	BookCorpus (4.5 GB)
GPT-2	2019	1.5 Mrd.	WebText (40 GB)
GPT-3	2020	175 Mrd.	CommonCrawl (570 GB), WebText2, Wikipedia, Books
GPT-4	2023	geschätzt 1.7 Bio.	nicht veröffentlicht
GPT-5	2025	nicht veröffentlicht	nicht veröffentlicht

3.2.3.5 DistilGPT2

DistilGPT2 ist eine kompakte Variante von GPT-2, die von Hugging Face mithilfe von Knowledge Distillation entwickelt wurde. Es basiert auf der gleichen Technik wie DistilBERT, das in Abschnitt 3.2.2.5 beschrieben ist. Das Modell verwendet die Architektur von GPT-2, wurde jedoch auf 82 Millionen Parameter reduziert und ist dadurch effizienter. In diesem Praxisteil kam DistilGPT2 anstelle von GPT-2 zum Einsatz, da es eine kürzere Trainingszeit auf der verfügbaren Hardware ermöglichte. Das Modell wurde für die englische Sprache trainiert und kann sowohl zur Textgenerierung als auch für Klassifikationsaufgaben verwendet werden [28].

4 Methodik und Umsetzung

Dieses Kapitel behandelt die praktische Umsetzung des Studienprojekts. Zunächst werden die verwendeten Datensätze und Bewertungsmetriken vorgestellt. Anschließend folgt die Implementierung und Evaluation der beiden Modelle. Zur Klassifikation wurden aus Effizienzgründen die komprimierten Modellvarianten `distilbert-base-uncased` und `distilgpt2` verwendet. Beide basieren auf den Architekturen von BERT bzw. GPT-2 und wurden jeweils auf zwei Datensätzen mit Phishing-Nachrichten angewendet.

4.1 Datensatzwahl

Für diese Arbeit werden zwei Datensätze verwendet, die in Textform vorliegen. Der Fokus liegt auf der inhaltlichen Analyse der Nachrichtentexte mithilfe von Methoden des NLP.

Der erste Datensatz stammt von Kaggle und umfasst rund 82.500 E-Mails, davon 42.891 Phishing- und 39.595 legitime Nachrichten [33]. Neben dem eigentlichen Nachrichtentext umfasst das zusammengeführte Textfeld auch die Betreffzeile, den Textkörper, das Datum sowie den Absender-E-Mail-Text.

Der zweite Datensatz stammt von Mendeley Data und enthält 5.971 SMS-Nachrichten, die als „Ham“ (legitim), „Spam“ oder „Smishing“ klassifiziert sind [14]. Er ist somit deutlich kleiner als der erste Datensatz. Verwendet wird ausschließlich der reine Nachrichtentext.

4.2 Evaluationsmetriken

Die Bewertung von Klassifikationsmodellen erfolgt typischerweise anhand einer Kontingenztabelle, welche die Klassifikationsentscheidungen in Bezug auf die tatsächliche Klassenzugehörigkeit darstellt [34]. Die einzelnen Ausprägungen der Klassifikation werden in Tabelle 4.1 dargestellt. Ausgehend davon lassen sich verschiedene Metriken berechnen, die jeweils unterschiedliche Aspekte des Models abbilden.

Tabelle 4.1: Kontingenztabelle

Bezeichnung	Bedeutung
True Positives (TP)	Fälle, die tatsächlich positiv sind und korrekt als positiv klassifiziert wurden.
False Positives (FP)	Fälle, die tatsächlich negativ sind, jedoch fälschlicherweise als positiv klassifiziert wurden.
False Negatives (FN)	Fälle, die tatsächlich positiv sind, jedoch fälschlicherweise als negativ klassifiziert wurden.
True Negatives (TN)	Fälle, die tatsächlich negativ sind und korrekt als negativ klassifiziert wurden.

4.2.1 Accuracy

Accuracy beschreibt den Anteil der korrekt klassifizierten Fälle (positiv und negativ) an der Gesamtanzahl der Fälle:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.1}$$

Eine hohe Accuracy bedeutet, dass das Modell insgesamt viele richtige Vorhersagen trifft. Bei stark unausgeglichenen Datensätzen wie beispielsweise bei wenigen Phishing-Fällen kann diese Metrik jedoch irreführend sein.

4.2.2 Precision

Precision misst die Genauigkeit unter den als positiv klassifizierten Fällen, also den Anteil der korrekt als Phishing erkannten Instanzen unter allen vom Modell als Phishing klassifizierten:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{4.2}$$

Eine hohe Precision bedeutet, dass Fehlalarme minimiert werden.

4.2.3 Recall

Recall beschreibt den Anteil der korrekt erkannten Phishing-Fälle unter allen tatsächlichen Phishing-Fällen:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4.3}$$

Ein hoher Recall bedeutet, dass nur wenige Phishing-Fälle übersehen werden.

4.2.4 F1-Score

Der F1-Score kombiniert Precision und Recall und ermöglicht eine zusammenfassende Bewertung der Modellleistung:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.4}$$

Der F1-Score ist insbesondere bei unausgeglichenen Datensätzen eine sinnvolle Gesamtkennzahl, da er sowohl die Fehlerquote (FP) als auch die Erkennungsrate (TP) einbezieht.

4.2.5 Zusammenfassung der Metriken

Die Accuracy (Gleichung 4.1) liefert eine erste Einschätzung des Modells, reicht in vielen Fällen jedoch nicht aus. Gerade bei unbalancierten Klassenverhältnissen sind Precision, Recall und der F_1-Score (Gleichungen 4.2–4.4) entscheidend für eine realistische Bewertung [34].

4.3 Modellimplementierungen

Für die Umsetzung der Phishing-Erkennung wurden mehrere Python-Notebooks entwickelt, die lokal auf einem System mit AMD Ryzen 7 5800X CPU und NVIDIA RTX 4080 GPU unter Nutzung von CUDA ausgeführt wurden. Im Folgenden werden Verarbeitungsschritte aufgeführt, die für alle Modellimplementierungen gelten, sofern nicht anders angegeben.

Der gesamte Quellcode ist im `GitHub-Repository` verfügbar.

4.3.1 Modelinitialisierung

Die Initialisierung der Modelle erfolgt mit der Transformers-Bibliothek von Hugging Face, die vortrainierte Sprachmodelle und zugehörige Tokenizer bereitstellt [35]. Für DistilBERT kann direkt ein Modell für Sequenzklassifikation geladen werden, wie in Codebeispiel 4.1 gezeigt. Hier ist bereits eine Klassifikationsschicht für binäre Aufgaben vorhanden.

```
tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased")

model = DistilBertForSequenceClassification.from_pretrained("distilbert-base
↪ -uncased",
↪  num_labels=2)
```

Code 4.1: Modelinitialisierung DistilBERT

Bei DistilGPT2 sind zusätzliche Schritte notwendig, wie in Codebeispiel 4.2 dargestellt. Da das Modell ursprünglich weder eine Klassifikationsschicht noch ein Padding-Token für die Eingabesequenzen enthält, wird das EOS-Token (End of Sequence) als Ersatz für Padding definiert. Anschließend wird das Modell mithilfe von `AutoModelForSequenceClassification` geladen, wodurch automatisch eine Klassifikationsschicht ergänzt wird.

```
tokenizer = GPT2Tokenizer.from_pretrained("distilgpt2")
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForSequenceClassification.from_pretrained("distilgpt2",
↪  num_labels=2)
model.config.pad_token_id = model.config.eos_token_id
```

Code 4.2: Modelinitialisierung DistilGPT2

4.3.2 Datenvorverarbeitung

Die Daten wurden mit `pandas` aus CSV-Dateien geladen und in ein `Dataset`-Objekt konvertiert, um sie direkt mit den Tokenizern weiterverarbeiten zu können. Die Tokenisierung erfolgte im Batch-Modus auf das gesamte Dataset. Je nach Modell kamen unterschiedliche Tokenizer zum Einsatz. Als Padding-Strategie wurde stets eine feste Sequenzlänge verwendet, sodass alle Eingaben modellunabhängig auf dieselbe Länge gebracht wurden. Anschließend wurden die Daten im Verhältnis 80:20 in Trainings- und Testdaten aufgeteilt. Codeausschnitt 4.3 zeigt den zugehörigen Code zum DistilGPT2-Modell.

```
def tokenize(batch):
    return tokenizer(batch["TEXT"], padding="max_length", truncation=True)

dataset = dataset.map(tokenize, batched=True)

split = dataset.train_test_split(test_size=0.2)
train_dataset = split["train"]
test_dataset = split["test"]
```

Code 4.3: Tokenisierung und Aufteilung in Trainings- und Testdaten

4.3.3 Trainingsparameter

Das Training erfolgte mit der `Trainer`-Klasse aus der Transformers-Bibliothek. Neben dem Modell und den Trainingsparametern wurden auch die Trainings- und Testdaten sowie eine Bewertungsfunktion übergeben, auf die in Abschnitt 4.3.4 näher eingegangen wird. Für beide Modelle wurde eine Batch-Größe von 16 beim ersten Datensatz und 8 beim zweiten Datensatz verwendet. Es wurden immer drei Trainings-Epochen durchgeführt. Beim GPT-Modell kam zusätzlich Mixed Precision zum Einsatz, um die Trainingsdauer zu reduzieren. Codeausschnitt 4.4 zeigt den zugehörigen Code zum DistilBERT-Modell.

```
training_args = TrainingArguments(
    output_dir="./path/to/output",
    per_device_train_batch_size=16,
    num_train_epochs=3,
    fp16=False,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

Code 4.4: Trainingsparameter und Modelltraining

4.3.4 Evaluierungsfunktion

Die zur Bewertung des Modells verwendete Funktion `compute_metrics` ist in Codebeispiel 4.5 dargestellt. Sie nutzt Funktionen der `scikit-learn`-Bibliothek zur Berechnung der in Abschnitt 4.2 beschriebenen Metriken Accuracy, Precision, Recall und F1-Score. Diese Werte dienen dazu, die Qualität der Modellvorhersagen objektiv zu bewerten und mit anderen Modellen vergleichen zu können. Die Funktion wird automatisch während der Evaluation aufgerufen und gibt die berechneten Kennzahlen in einem Dictionary zurück.

```
def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds,
    ↪  average="binary")
    acc = accuracy_score(labels, preds)
    return {
        "accuracy": acc,
        "f1": f1,
        "precision": precision,
        "recall": recall
}
```

Code 4.5: Berechnung der Evaluationsmetriken

Zusätzlich wird für alle Modelle während des Trainings der Loss-Wert berechnet. Diese Kennzahl gibt an, wie stark die Vorhersagen von den tatsächlichen Ergebnissen abweichen, und wird im Laufe des Trainings minimiert. Ein niedriger Loss-Wert deutet darauf hin, dass das Modell die Zusammenhänge in den Trainingsdaten gut erfasst hat. Anschließend wird auf Basis der vorhergesagten und tatsächlichen Klassen eine Confusion-Matrix erstellt, die visualisiert, wie viele Beispiele korrekt oder fehlerhaft eingeordnet wurden. Besonders relevant sind dabei False Positives (FP) und False Negatives (FN), insbesondere bei sicherheitskritischen Anwendungen wie der Phishing-Erkennung.

4.3.5 DistilBERT zur Erkennung von Mail-Phishing

Im Folgenden wird das DistilBERT-Modell auf den in Abschnitt 4.1 beschriebenen Mail-Datensatz [33] angewendet und evaluiert.

4.3.5.1 Training des Modells

Während des Trainings wurde der Loss-Wert über die Trainingsepochen hinweg aufgezeichnet. Abbildung 4.1 zeigt den Verlauf des Trainings-Loss über drei Epochen hinweg.

Die gesamte Trainingszeit betrug 2088,62 Sekunden, was etwa 34,81 Minuten entspricht.

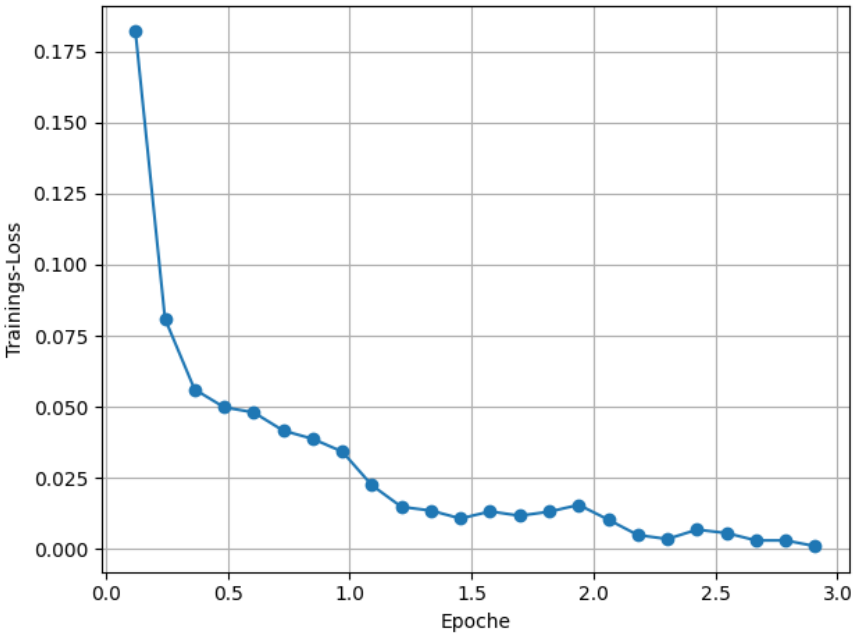

Abbildung 4.1: Trainings-Loss (DistilBERT, Mail-Phishing)

4.3.5.2 Evaluation des Modells

Tabelle 4.2 fasst die beim Testen des Modells berechneten Metriken und Zeitwerte zusammen und bietet eine Übersicht über die wichtigsten Kennzahlen.

Tabelle 4.2: Evaluationsmetriken (DistilBERT, Mail-Phishing)

Metrik	**Ergebnis**
Accuracy	0,9951
Precision	0,9941
Recall	0,9965
F1-Score	0,9953
Tokenisierungsdauer	319,99 Sekunden (5,33 Minuten)
Trainingsdauer	2088,62 Sekunden (34,81 Minuten)
Evaluationsdauer	54,63 Sekunden (0,91 Minuten)

Zur besseren Visualisierung wurde zusätzlich eine Confusion-Matrix erstellt. Abbildung 4.2 zeigt die Anzahl korrekt und falsch klassifizierter Instanzen beider Klassen.

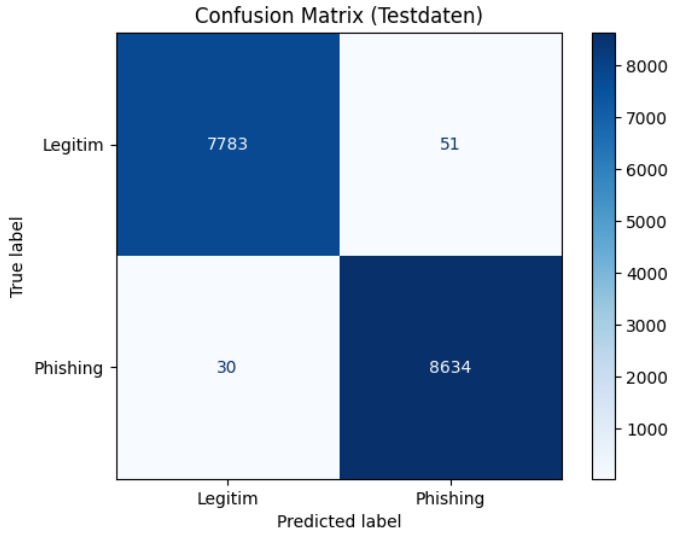

Abbildung 4.2: Confusion-Matrix (DistilBERT, Mail-Phishing)

4.3.5.3 Fazit

Die Ergebnisse zeigen, dass das DistilBERT-Modell eine sehr hohe Erkennungsgenauigkeit auf dem Mail-Phishing-Datensatz erzielt. Mit einer Accuracy von 0,9951 und einem F1-Score von 0,9953 klassifiziert es legitime und phishingverdächtige E-Mails im Testdatensatz nahezu fehlerfrei. Die in Abbildung 4.2 gezeigte Confusion-Matrix unterstreicht die Modellleistung.

4.3.6 DistilBERT zur Erkennung von SMS-Phishing

Im Folgenden wird das DistilBERT-Modell auf den in Abschnitt 4.1 beschriebenen SMS-Datensatz [14] angewendet und evaluiert.

4.3.6.1 Training des Modells

Während des Trainings wurde der Loss-Wert über die Trainingsepochen hinweg aufgezeichnet. Abbildung 4.3 zeigt den Verlauf des Trainings-Loss über drei Epochen hinweg.

Die gesamte Trainingszeit betrug 204,46 Sekunden, was etwa 3,41 Minuten entspricht.

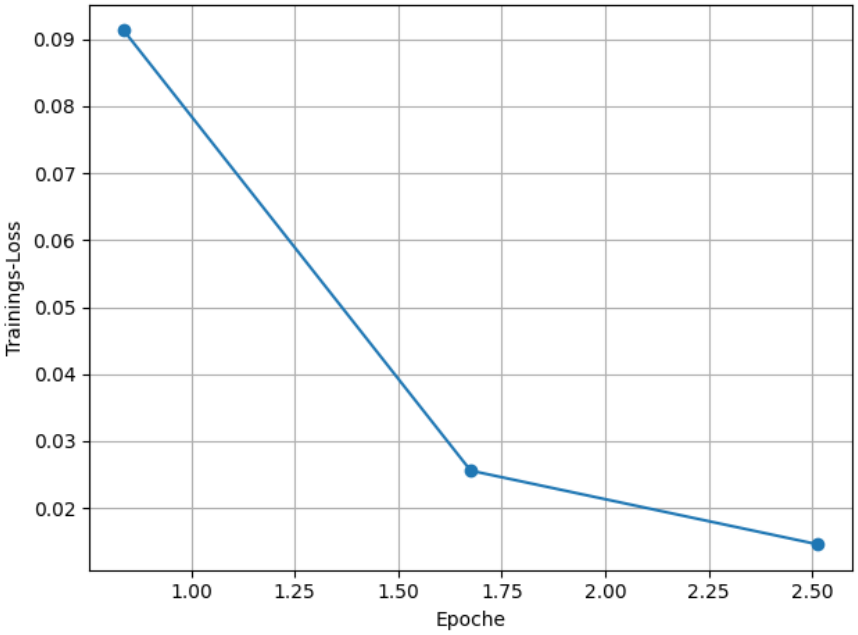

Abbildung 4.3: Trainings-Loss (DistilBERT, SMS-Phishing)

4.3.6.2 Evaluation des Modells

Tabelle 4.3 fasst die beim Testen des Modells berechneten Metriken und Zeitwerte zusammen und bietet eine Übersicht über die wichtigsten Kennzahlen.

Tabelle 4.3: Evaluationsmetriken (DistilBERT, SMS-Phishing)

Metrik	**Ergebnis**
Accuracy	0,9933
Precision	0,9817
Recall	0,9817
F1-Score	0,9817
Tokenisierungsdauer	2,03 Sekunden (0,03 Minuten)
Trainingsdauer	204,46 Sekunden (3,41 Minuten)
Evaluationsdauer	3,9 Sekunden (0,07 Minuten)

Zur besseren Visualisierung wurde zusätzlich eine Confusion-Matrix erstellt. Abbildung 4.4 zeigt die Anzahl korrekt und falsch klassifizierter Instanzen beider Klassen.

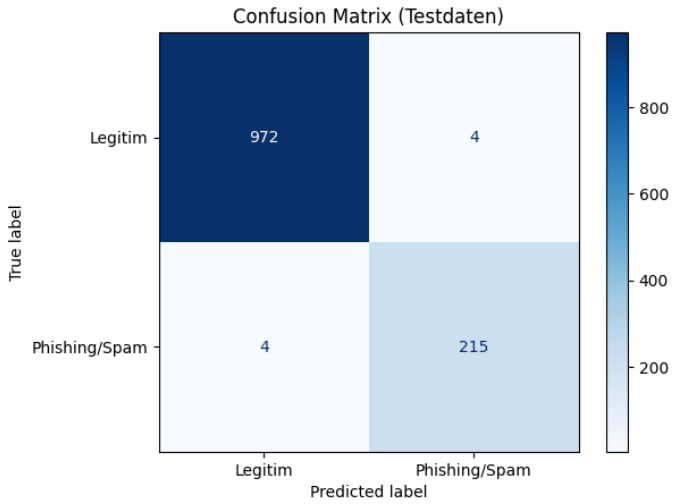

Abbildung 4.4: Confusion-Matrix (DistilBERT, SMS-Phishing)

4.3.6.3 Fazit

Auch auf dem SMS-Phishing-Datensatz zeigt das DistilBERT-Modell eine überzeugende Leistung. Es erreicht eine Accuracy von 0,9933 sowie einen F1-Score von 0,9817 und trennt legitime von Phishing- bzw. Spam-Nachrichten zuverlässig. Die Confusion-Matrix in Abbildung 4.4 veranschaulicht diese Ergebnisse.

4.3.7 DistilGPT2 zur Erkennung von Mail-Phishing

Im Folgenden wird das DistilGPT2-Modell auf den in Abschnitt 4.1 beschriebenen Mail-Datensatz [33] angewendet und evaluiert.

4.3.7.1 Training des Modells

Während des Trainings wurde der Loss-Wert über die Trainingsepochen hinweg aufgezeichnet. Abbildung 4.5 zeigt den Verlauf des Trainings-Loss über drei Epochen hinweg.

Die gesamte Trainingszeit betrug 3601,23 Sekunden, was etwa 60,02 Minuten entspricht.

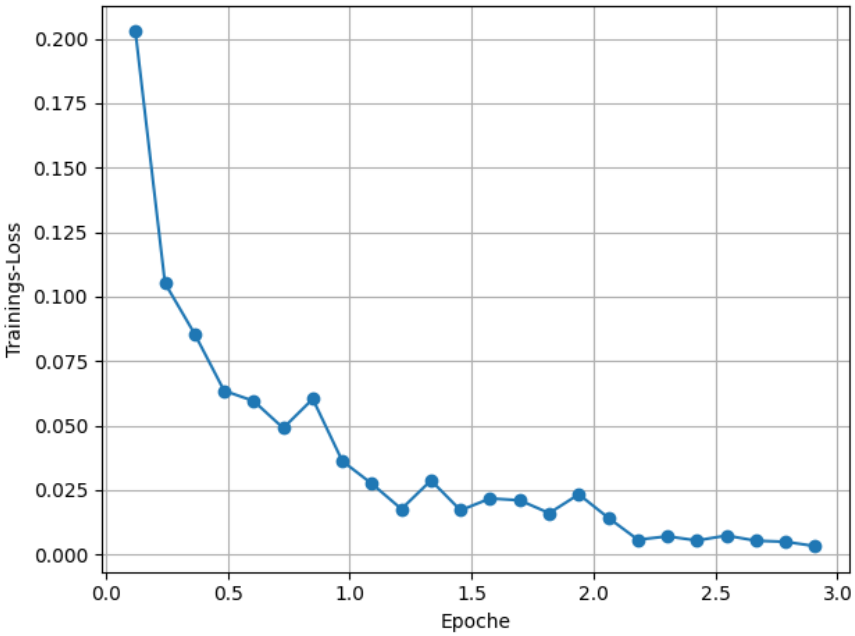

Abbildung 4.5: Trainings-Loss (DistilGPT2, Mail-Phishing)

4.3.7.2 Evaluation des Modells

Tabelle 4.4 fasst die beim Testen des Modells berechneten Metriken und Zeitwerte zusammen und bietet eine Übersicht über die wichtigsten Kennzahlen.

Tabelle 4.4: Evaluationsmetriken (DistilGPT2, Mail-Phishing)

Metrik	**Ergebnis**
Accuracy	0,9953
Precision	0,9975
Recall	0,9935
F1-Score	0,9955
Tokenisierungsdauer	121,17 Sekunden (2,02 Minuten)
Trainingsdauer	3601,23 Sekunden (60,02 Minuten)
Evaluationsdauer	99,94 Sekunden (1,67 Minuten)

Zur besseren Visualisierung wurde zusätzlich eine Confusion-Matrix erstellt. Abbildung 4.6 zeigt die Anzahl korrekt und falsch klassifizierter Instanzen beider Klassen.

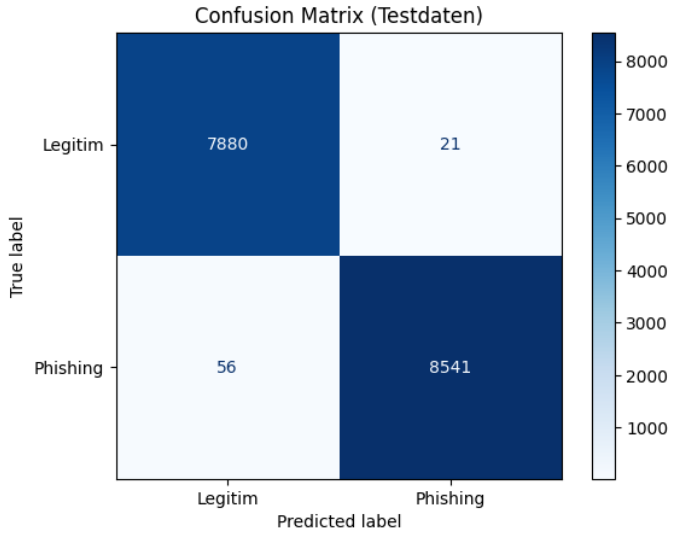

Abbildung 4.6: Confusion-Matrix (DistilGPT2, Mail-Phishing)

4.3.7.3 Fazit

Das DistilGPT2-Modell erzielt eine sehr hohe Erkennungsgenauigkeit auf dem Mail-Phishing-Datensatz. Mit einer Accuracy von 0,9953 und einem F1-Score von 0,9955 klassifiziert es legitime und phishingverdächtige E-Mails fast fehlerfrei. Die Trainingsdauer ist mit 60,02 Minuten jedoch fast doppelt so hoch wie beim DistilBERT-Modell. Die in Abbildung 4.2 gezeigte Confusion-Matrix unterstreicht die Modellleistung.

4.3.8 DistilGPT2 zur Erkennung von SMS-Phishing

Im Folgenden wird das DistilGPT2-Modell auf den in Abschnitt 4.1 beschriebenen SMS-Datensatz [14] angewendet und evaluiert.

4.3.8.1 Training des Modells

Während des Trainings wurde der Loss-Wert über die Trainingsepochen hinweg aufgezeichnet. Abbildung 4.7 zeigt den Verlauf des Trainings-Loss über drei Epochen hinweg.

Die gesamte Trainingszeit betrug 289,49 Sekunden, was etwa 4,82 Minuten entspricht.

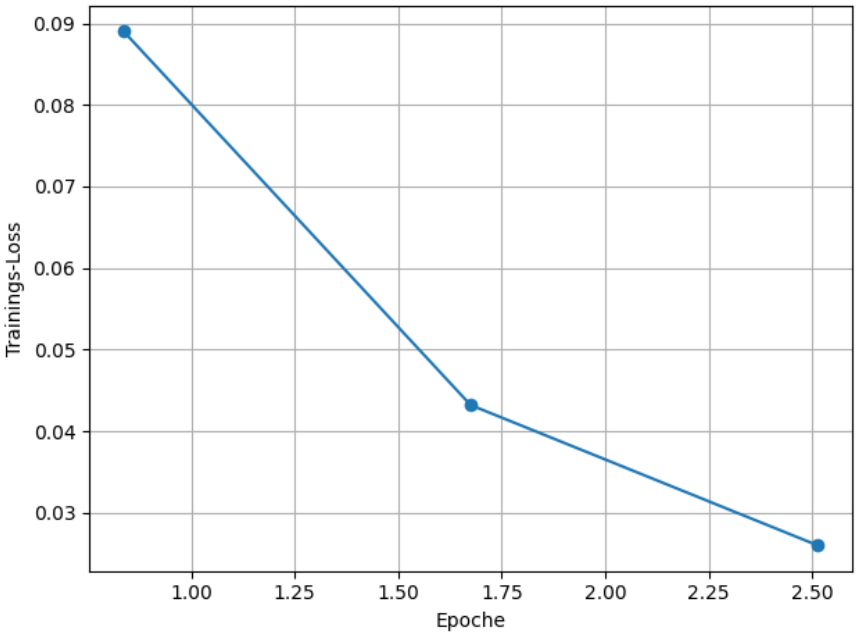

Abbildung 4.7: Trainings-Loss (DistilGPT2, SMS-Phishing)

4.3.8.2 Evaluation des Modells

Tabelle 4.5 fasst die beim Testen des Modells berechneten Metriken und Zeitwerte zusammen und bietet eine Übersicht über die wichtigsten Kennzahlen.

Tabelle 4.5: Evaluationsmetriken (DistilGPT2, SMS-Phishing)

Metrik	**Ergebnis**
Accuracy	0,9941
Precision	0,9952
Recall	0,9721
F1-Score	0,9835
Tokenisierungsdauer	2,55 Sekunden (0,04 Minuten)
Trainingsdauer	289,49 Sekunden (4,82 Minuten)
Evaluationsdauer	8,68 Sekunden (0,14 Minuten)

Zur besseren Visualisierung wurde zusätzlich eine Confusion-Matrix erstellt. Abbildung 4.8 zeigt die Anzahl korrekt und falsch klassifizierter Instanzen beider Klassen.

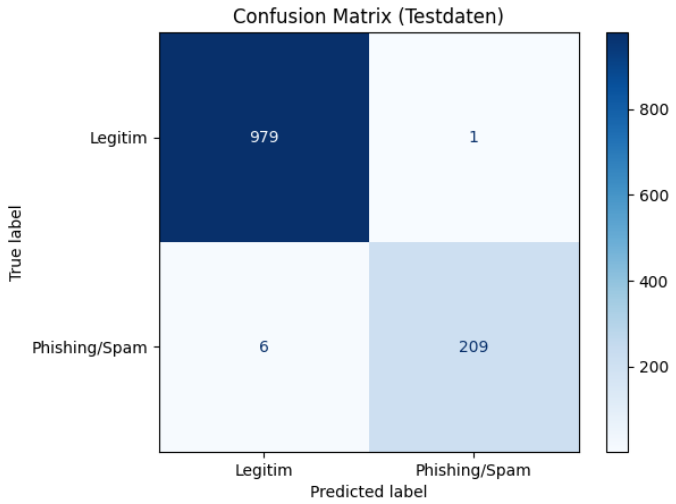

Abbildung 4.8: Confusion-Matrix (DistilGPT2, SMS-Phishing)

4.3.8.3 Fazit

Auch auf dem SMS-Phishing-Datensatz zeigt das DistilGPT2-Modell eine sehr gute Leistung. Es erreicht eine Accuracy von 0,9941 sowie einen F1-Score von 0,9835 und trennt legitime von Phishing- bzw. Spam-Nachrichten zuverlässig. Die Confusion-Matrix in Abbildung 4.4 veranschaulicht diese Ergebnisse.

5 Fazit und Ausblick

Diese Arbeit bietet einen umfassenden Einblick in unterschiedliche Formen und Merkmale von Phishing, einschließlich psychologischer, sprachlicher und technischer Aspekte. Es wird dargestellt, wie Phishing-Angriffe aufgebaut sein können, welche Manipulationsstrategien verwendet werden und welche Herausforderungen dies für die automatische Erkennung bedeutet. Durch die Unterscheidung zwischen klassischen Phishing-Mails, Smishing, Vishing und gezieltem Spear-Phishing wird die Bedrohungslage differenziert betrachtet.

Neben den Grundlagen zu Phishing wird auch die Entwicklung des NLP behandelt. Nach einer Einführung in regelbasierte und statistische Verfahren werden moderne Ansätze aus dem Bereich der künstlichen Intelligenz vorgestellt. Es wird erläutert, wie aktuelle Modelle Wörter verarbeiten und interpretieren, welche Rolle Embeddings dabei spielen und wie die grundlegende Architektur von Transformern funktioniert. Der Fokus liegt dabei auf den beiden bekanntesten Vertretern BERT und GPT sowie deren Unterschieden.

Im praktischen Teil liegt der Schwerpunkt auf dem konkreten Einsatz von BERT und GPT zur Erkennung von Phishing-Nachrichten. Nach Vorstellung der verwendeten Datensätze und Evaluationsmetriken werden beide Modelle gezielt für Klassifikationsaufgaben fine-getuned. Die Tests auf zwei unterschiedlichen Datensätzen, bestehend aus E-Mail-Phishing-Nachrichten und Smishing-Beispielen, zeigen sehr gute Ergebnisse. Trotzdem gibt es klare Einschränkungen. Die Modelle arbeiten gut auf simplen Beispielen. Komplexe Angriffe wie Spear-Phishing oder langfristige Social-Engineering-Kampagnen lassen sich mit den verwendeten Datensätzen nicht abbilden.

Langfristig stellt sich außerdem die Frage, wie sich solche Systeme in reale E-Mail-Infrastrukturen integrieren lassen, möglicherweise auch in Kombination mit traditionellen Verfahren. Auch die Nachvollziehbarkeit der Entscheidungen mittels Explainable Artificial Intelligence (XAI) könnte dabei eine wichtige Rolle spielen. Trotz der vielseitigen Anwendungsmöglichkeiten moderner Sprachmodelle bleibt das Verständnis ihrer internen Mechanismen schwierig und ähnelt aufgrund der hohen Komplexität oft einer „Blackbox“. XAI-Methoden können hier ansetzen, indem sie die Entscheidungswege transparent machen, die für die Klassifikation ausschlaggebenden Wörter oder Merkmale hervorheben und den Prozess dadurch nutzerfreundlicher gestalten [36].

Für zukünftige Arbeiten wäre es sinnvoll, stärker mit kontextbasierten Datensätzen zu arbeiten, insbesondere im Bereich Spear-Phishing. Dabei sollte untersucht werden, wie sich komplexe Phishing-Angriffe abbilden lassen, die über längere Kommunikationsverläufe hinweg stattfinden. Dazu gehören etwa Angriffe, bei denen eine Beziehung zwischen Angreifer und Opfer über mehrere Nachrichten hinweg aufgebaut wird. Um solche Szenarien zu erfassen, müssten neben dem reinen Nachrichtentext auch Metadaten, Antwortverhalten und Absenderinformationen berücksichtigt werden.

Darüber hinaus wäre es sinnvoll, weitere Modelle zu testen. Dadurch ließe sich besser einschätzen, welchen Einfluss die Modellarchitektur und -größe auf die Leistung hat und ob kleinere, ressourcenschonendere Modelle bereits ausreichen. In diesem Zusammenhang wäre auch zu klären, wie groß ein Datensatz mindestens sein muss, um verlässliche Ergebnisse zu erzielen, und wo eine sinnvolle Grenze zwischen Modellkomplexität, Trainingsaufwand und Genauigkeit liegt, um befriedigende Resultate zu erreichen.

Literaturverzeichnis

[1] Shahla Eltayeb. „Unlocking the Human Element in Cybercrime: Exploring the Realities of Victims“. In: *Cybercrime Unveiled: Technologies for Analysing Legal Complexity*. Hrsg. von Mohamed Chawki und Ajith Abraham. Cham: Springer Nature Switzerland, 2025, S. 27–29. DOI: 10.1007/978-3-031-80557-8_2.

[2] Bundesamt für Verfassungsschutz. *Vorstellung der Bitkom-Studie „Wirtschaftsschutz 2024“*. 2024. URL: https://www.verfassungsschutz.de/SharedDocs/kurzmeldungen/DE/2024/2024-08-28-studie-bitkom.html (besucht am 2. Mai 2025).

[3] Bitkom e.V. *Cybercrime-Bilanz: 6 von 10 Internetnutzern sind betroffen*. 2025. URL: https://www.bitkom.org/Presse/Presseinformation/Cybercrime-Bilanz-6-von-10-Internetnutzern-betroffen (besucht am 2. Mai 2025).

[4] Eva-Maria Weiß. *FraudGPT schreibt Phishing-Mails und entwickelt Malware*. 2023. URL: https://www.heise.de/news/FraudGPT-schreibt-Phishing-Mails-und-entwickelt-Malware-9231555.html (besucht am 30. Apr. 2025).

[5] Said Salloum u. a. „Phishing Email Detection Using Natural Language Processing Techniques: A Literature Survey“. In: *Procedia Computer Science* 194 (2021), S. 19–28. DOI: 10.1016/j.procs.2021.05.077.

[6] Bundesamt für Sicherheit in der Informationstechnik. *Spam, Phishing & Co*. URL: https://www.bsi.bund.de/dok/507910 (besucht am 3. Mai 2025).

[7] Richard Adolph Aires Jonker u. a. „Using Natural Language Processing for Phishing Detection“. In: *Optimization, Learning Algorithms and Applications*. Hrsg. von Ana I. Pereira u. a. Cham: Springer International Publishing, 2021, S. 540–552. DOI: 10.1007/978-3-030-91885-9_40.

[8] Anjuli Franz und Alexander Benlian. „Spear Phishing 2.0: Wie automatisierte Angriffe Organisationen vor neue Herausforderungen stellen“. In: *HMD Praxis der Wirtschaftsinformatik* 57 (2020), S. 597–612. DOI: 10.1365/s40702-020-00613-y.

[9] Fortinet. *Welche Arten von Phishing gibt es?* URL: https://www.fortinet.com/de/resources/cyberglossary/types-of-phishing-attacks (besucht am 29. Mai 2025).

[10] Bundesamt für Sicherheit in der Informationstechnik. *Social Engineering – der Mensch als Schwachstelle*. URL: https://www.bsi.bund.de/dok/11287460 (besucht am 3. Juni 2025).

[11] Polizeiliche Kriminalprävention der Länder und des Bundes (ProPK). *Vorsicht vor falschen Enkeln*. URL: https://www.polizei-beratung.de/themen-und-tipps/betrug/enkeltrick/ (besucht am 3. Juni 2025).

[12] Michael Workman. „Wisecrackers: A theory-grounded investigation of phishing and pretext social engineering threats to information security". In: *Journal of the American Society for Information Science and Technology* 59.4 (2008), S. 662–674. DOI: https://doi.org/10.1002/asi.20779.

[13] Robert B. Cialdini. *Influence: Science and Practice*. 5. Aufl. Boston: Pearson Education, 2009. ISBN: 978-0-205-60999-4.

[14] Sandhya Mishra und Devpriya Soni. *SMS Phishing Dataset for Machine Learning and Pattern Recognition*. Version V1. Mendeley Data, 2022. DOI: 10.17632/f45bkkt8pr.1.

[15] Tarini Saka u. a. *Phishing Codebook: A structured framework for the characterization of phishing emails*. WorkingPaper. ArXiv, 2024, S. 1–18. DOI: 10.48550/arXiv.2408.08967.

[16] Moshood Abiola Polytechnic u. a. „Ai-Powered Phishing Detection And Prevention". In: *Path of Science* 10 (2024), S. 4001–4010. DOI: 10.22178/pos.112-7.

[17] B. B. Gupta u. a. „Fighting against phishing attacks: state of the art and future challenges". In: *Neural Computing and Applications* 28 (2017), S. 3629–3654. DOI: 10.1007/s00521-016-2275-y.

[18] Apurv Mittal u. a. „Phishing Detection Using Natural Language Processing and Machine Learning". In: *SMU Data Science Review* 6 (2022). URL: https://scholar.smu.edu/datasciencereview/vol6/iss2/14.

[19] Prashant Johri u. a. „Natural Language Processing: History, Evolution, Application, and Future Work". In: Jan. 2021, S. 365–375. DOI: 10.1007/978-981-15-9712-1_31.

[20] John Hutchins. *The history of machine translation in a nutshell*. 2014. URL: https://aclanthology.org/www.mt-archive.info/10/Hutchins-2014.pdf.

[21] Jacob Devlin u. a. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[22] Daniel Jurafsky und James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. Bd. 3. Online manuscript released January 12, 2025. 2025. URL: https://web.stanford.edu/~jurafsky/slp3/.

[23] Jason W. Wei und Kai Zou. „EDA: Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks“. In: *CoRR* abs/1901.11196 (2019). arXiv: 1901.11196. URL: http://arxiv.org/abs/1901.11196.

[24] John's AI Group. *Embeddings Walkthrough Slide*. 2018. URL: https://github.com/tanchongmin/strictjson/blob/main/Experiments/Embeddings%20Walkthrough.pdf.

[25] Ashish Vaswani u. a. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[26] OpenAI. *Introducing ChatGPT*. 2022. URL: https://openai.com/blog/chatgpt (besucht am 24. Juli 2025).

[27] Yinhan Liu u. a. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. arXiv: 1907.11692 [cs.CL]. URL: https://arxiv.org/abs/1907.11692.

[28] Victor Sanh u. a. *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. 2020. arXiv: 1910.01108 [cs.CL]. URL: https://arxiv.org/abs/1910.01108.

[29] Zhenzhong Lan u. a. *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. 2020. arXiv: 1909.11942 [cs.CL]. URL: https://arxiv.org/abs/1909.11942.

[30] Alec Radford u. a. „Improving Language Understanding by Generative Pre-Training“. In: (2018). Preprint. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

[31] Alec Radford u. a. „Language Models are Unsupervised Multitask Learners“. In: (2019). Technical Report. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[32] Wikipedia contributors. *Products and applications of OpenAI*. 2025. URL: https://en.wikipedia.org/wiki/Products_and_applications_of_OpenAI (besucht am 29. Aug. 2025).

[33] Naser Abdullah Alam. *Phishing Email Dataset*. 2024. DOI: 10.34740/KAGGLE/DS/5074342. URL: https://www.kaggle.com/ds/5074342.

[34] Claude Sammut und Geoffrey Webb. *Encyclopedia of Machine Learning*. 2010. DOI: 10.1007/978-0-387-30164-8.

[35] Hugging Face. *Hugging Face – The AI community building the future*. 2025. URL: https://huggingface.co (besucht am 24. Mai 2025).

[36] Mohammad Amaz Uddin, Md Mahiuddin und Iqbal H. Sarker. *An Explainable Transformer-based Model for Phishing Email Detection: A Large Language Model Approach*. 2025. arXiv: 2402.13871 [cs.LG]. URL: https://arxiv.org/abs/2402.13871.